

TRIBHUVAN UNIVERSITY

FACULTY OF HUMANITIES AND SOCIAL SCIENCE

A Project Report

On

"Interconnected: A Social Media Platform"

Submitted to

Department of Computer Application

Prime College

In partial fulfillement of the requriments for Bachelor Degree in Computer Application

Submitted By:

Utsab Dangol

(41002190)

January,2026

Under the Supervision of

Mr. Radha Krishna Gajurel



TRIBHUVAN UNIVERSITY

FACULTY OF HUMANITIES AND SOCIAL SCIENCE

SUPERVISOR'S RECOMMENDATION

The project work report entitled "**Interconnected: A Web-Based Community Platform**" submitted by Utsab Dangol of Prime College, Khusibu, Kathmandu, is prepared under my supervision as per the procedure and format laid by the Faculty of Humanities and Social Science, Tribhuvan University, as partial fulfillment for the degree of Bachelors in Computer Application (BCA). I, therefore, recommend the project work for future evaluation.

Mr. Radha Krishna Gajurel

Supervisor

Prime College



TRIBHUVAN UNIVERSITY

FACULTY OF HUMANITIES AND SOCIAL SCIENCE

LETTER OF APPROVAL

This is to certify that this project prepared by Utsab Dangol entitled "**Interconnected: A Web-Based Community Platform**" in partial fulfillment of the requirements for the Degree of Bachelor's in Computer Application has been evaluated. In our opinion it is satisfactory in scope and quality as a project for the required degree.

SIGNATURE of Supervisor **Mr. Radha Krishna Gajurel** **Prime College**	**SIGNATURE of Internal Examiner/ Coordinator** **Er. Rolisha Sthapit** **Prime College**
	SIGNATURE of External Examiner **Mr. Kumar Prasun**

ABSTRACT

Interconnected is a comprehensive social media platform designed to facilitate community-driven interaction and content sharing. The system enables users to create and join specific interest-based communities, fostering focused discussions and networking. Users can engage by sharing multimedia posts, participating in threaded comment sections, and utilizing appreciation mechanisms like likes. The platform incorporates a robust moderation system allowing administrators to manage communities, review user reports, and maintain platform integrity. By offering a structured environment for social exchange, Interconnected addresses the need for niche networking spaces where users can connect over shared passions. The application leverages a modern technology stack to ensure performance and scalability, providing a seamless user experience for content consumption and interaction. Overall, Interconnected provides a user-friendly, secure, and dynamic solution for building and sustaining online communities.

Keywords: Interconnected, Community Platform, Social Media, PHP, MySQL, React, Content Management, content Moderation, Web Architecture.

ACKNOWLEDGEMENT

This report on **"Interconnected: A Web-Based Community Platform"** has been completed with great effort and dedication. However, it would not have been possible without the valuable support and guidance of many individuals who contributed to its success. I would like to express my heartfelt gratitude to all of them. I am deeply thankful to Prime College and our respected teacher **Mr. Radha Krishna Gajurel** for providing me with this valuable opportunity to apply my theoretical knowledge in a practical field. Their guidance, supervision, and continuous support have been instrumental in the successful completion of this report. Finally, I would like to extend my sincere appreciation to all those who have willingly shared their time, knowledge, and assistance throughout this project. Their encouragement and cooperation have been a great source of motivation for me.

Prepared by
Utsab Dangol
Bachelors of Computer Application
Prime College

TABLE OF CONTENT

SUPERVISOR'S RECOMMENDATION ..... **i**
LETTER OF APPROVAL ..... **ii**
ABSTRACT ..... **iii**
ACKNOWLEDGEMENT ..... **iv**
TABLE OF CONTENT ..... **v**
ABBREVIATIONS ..... **vii**
LIST OF FIGURES ..... **viii**
LIST OF TABLES ..... **ix**
CHAPTER 1 INTRODUCTION **1**
1.1 Introduction 1
1.2 Problem statement 2
1.3 Objectives 2
1.4 scope and limitation 3
1.4.1 Scope 3
1.4.2 Limitations 3
1.5 Development Methodology 4
1.6 Report Organization 7
CHAPTER 2 BACKGROUND STUDY AND LITERATURE REVIEW **8**
2.1 Background Study 8
2.2 Literature Review 9
CHAPTER 3 SYSTEM ANALYSIS AND DESIGN **11**
3.1 System Analysis 11
3.1.1 Requirement Analysis 11
3.1.2 Feasibility Study 13
3.1.3 Data Modeling 16
3.1.4 Process Modeling 18
3.2 System Design 21
3.2.1 System Architecture 21
3.2.2 Database Schema Design 22
3.2.3 Interface Design (UI Interface/ Interface Structure Diagrams) 24
3.2.4 Physical DFD 30
CHAPTER 4 IMPLEMENTATION AND TESTING **31**

4.1 Implementation 31
4.1.1 Tools Used 31
4.1.2 Implementation of Modules 32
4.2 Testing 37
4.2.1 Unit Testing 37
4.2.2 System Testing 40
CHAPTER 5 CONCLUSION AND FUTURE RECOMMENDATIONS 42
5.1 Lessons Learnt/Outcome 42
5.2 Conclusion 43
5.3 Future Recommendations 44
REFERENCES 46
APPENDICES

ABBREVIATIONS

API	Application Programming Interface
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
DFD	Data Flow Diagram
ERD	Entity Relationship Diagram
HTML	HyperText Markup Language
HTTPS	HyperText Transfer Protocol Secure
KPI	Key Performance Indicator
MySQL	My Structured Query Language
PHP	Hypertext Preprocessor
SDLC	Software Development Life Cycle
SQL	Structured Query Language
UI	User Interface
XAMPP	Cross-Platform (X), Apache, MariaDB/MySQL, PHP, Perl

LIST OF FIGURES

Figure 1.1: Waterfall Model of Interconnected Platform....4
Figure 3.1: Use Case Diagram of Interconnected Platform....12
Figure 3.2 Gantt Chart of interconnected Platform....15
Figure 3.3: Entity Relationship Diagram of Interconnected Platform....17
Figure 3.4: Context Level Diagram of Interconnected Platform....18
Figure 3.5: User Level 1 Data Flow Diagram of Interconnected Platform....19
Figure 3.6: Admin Level 1 Data Flow Diagram of Interconnected Platform....20
Figure 3.7: System Architecture of Interconnected Platform....22
Figure 3.8: Database Schema Design of Interconnected Platform....22
Figure 3.9: Interface Design of Interconnected Platform....24
Figure 3.10: Low Fidelity Design Registration Page....25
Figure 3.11: High Fidelity Design Registration Page....25
Figure 3.12: Low Fidelity Design Home Page....26
Figure 3.13: High Fidelity Design Home Page....26
Figure 3.14: Low Fidelity Design Profile Page....27
Figure 3.15: High Fidelity Design Profile Page....27
Figure 3.16: Low Fidelity Design Communities Page....28
Figure 3.17: High Fidelity Design Communities Page....28
Figure 3.18: Low Fidelity Design Admin Page....29
Figure 3.19: Low Fidelity Design Admin Page....29
Figure 3.20: Physical Diagram of Interconnected Platform....30

LIST OF TABLES

Table 3.1: Gantt Table of interconnected Platform 16
Table 4.1: Tool used 31
Table 4.2: User Registration Testing 37
Table 4.3: Community Management Testing 38
Table 4.4: Content Posting Testing 39
Table 4.5: User Interface Testing 40
Table 4.6: Admin Interface Testing 41

CHAPTER 1
INTRODUCTION

1.1 Introduction

Social media has transformed how people communicate, but most platforms now optimize for endless engagement rather than genuine connection. The heavy use of algorithmic feeds and addictive design often overwhelms users with constant noise, leading to shallow interactions and fragmented attention.

Interconnected aims to counter this trend by creating a community-first social platform where people can focus on meaningful conversations with others who share their interests. Instead of pushing algorithm-driven feeds, the system uses a single-community viewing model that highlights depth over distraction. Users can join many communities, but only one is shown at a time, encouraging intentional, focused engagement.

This design is built on a simple idea: people communicate better when they aren't bombarded with unrelated content. By reducing information overload and emphasizing topic-specific spaces, Interconnected supports clearer thinking, richer discussion, and a healthier online experience.

Interconnected serves multiple user groups including community creators who want to build focused discussion spaces around specific topics, community members seeking meaningful interactions with others who share their interests, and moderators who maintain the quality and relevance of community discussions. The platform provides comprehensive tools for community management, content creation and curation, user interaction, and administrative oversight.

Interconnected brings together three kinds of people: creators who build focused discussion spaces, members who want meaningful conversations, and moderators who keep communities healthy. The platform gives each group the tools they need for managing communities, creating content, and maintaining quality.

It's built with React.js for a fast, dynamic interface, PHP for the core application logic, and MySQL for stable data management. The system uses a clean three-tier architecture separating the UI, backend logic, and database to keep everything scalable, secure, and easy to maintain.

Interconnected shows that social platforms don’t have to rely on addictive design or algorithmic manipulation. By centering communities and giving users control over their interactions, it creates a space for focused, productive, and truly meaningful online engagement.

1.2 Problem statement

Social media has become a big part of modern communication and community building. However, current platforms often prioritize engagement-driven algorithms that can lead to distraction and superficial interactions. While platforms like Facebook offer community features, they are often overshadowed by algorithmic feeds that promote irrelevant content. Apps like Pinterest excel at idea sharing but lack meaningful community interaction, and Discord focuses on group communication but doesn't emphasize productive, interest-based connections.

Most existing social media platforms rely heavily on engagement-driven algorithms that tend to promote controversial or trending content rather than material that is relevant or productive. As a result, users are frequently distracted and find it difficult to maintain focus on meaningful topics. Although community features are present on platforms such as Facebook, they are not treated as the central element of the user experience and are often difficult to access or navigate efficiently. In addition, these platforms rarely achieve a healthy balance between entertainment and productivity, usually prioritizing one at the expense of the other. Interactions are often limited to quick reactions or short comments, which discourages thoughtful discussion and weakens genuine user connections. Furthermore, the constant flow of content from multiple sources overwhelms users, making it challenging for them to concentrate on specific interests or actively engage within a single community.

These issues create a need for a platform that facilitates meaningful connections between users with shared interests while promoting productive engagement and reducing digital distraction.

1.3 Objectives

The primary objectives of developing Interconnected are:

- To create a community-driven social media platform that prioritizes meaningful connections and productive interactions among users with shared interests.

- To balance social interaction with productivity by reducing algorithmic distractions

1.4 scope and limitation

1.4.1 Scope

The Interconnected platform is designed to support community-oriented social networking. Users will be able to register securely, create and manage personal profiles, and participate in communities aligned with their interests. Each community will function as a dedicated space for topic-focused discussions, enabling users to create, edit, and engage with posts and threaded comments. Community creators and moderators will have the necessary tools to manage members, oversee content, and enforce community guidelines.

The platform includes administrative controls that allow system administrators to supervise user activity, review reported content, and manage communities through a centralized dashboard. The technical implementation employs React.js for the frontend interface, PHP for backend processing, and MySQL for structured data storage. The system architecture is developed with scalability in mind, ensuring stable performance as user activity and content volume increase.

1.4.2 Limitations

The initial version of the platform prioritizes core community and discussion features. Advanced functionalities such as real-time messaging, video sharing, live streaming, and extensive multimedia support are not included. Access is limited to web browsers, with no native mobile applications or offline capabilities.

File uploads are restricted in both size and supported formats, emphasizing text-based content with limited image support. Scalability constraints may arise due to the absence of advanced infrastructure components such as content delivery networks, distributed caching, and load balancing.

Moderation depends primarily on human review, with no integrated artificial intelligence or automated content-analysis systems. The platform supports only English in its first release, lacking multilingual interfaces or localization features. Additionally, advanced analytics, detailed behavioral insights, and integration with external services such as social logins or cross-platform content sharing are beyond the scope of the current implementation.

1.5 Development Methodology

For the Interconnected platform, the Waterfall Model was adopted. This approach is appropriate because the project requirements were largely identified and finalized prior to development. The model's linear and sequential structure allows each phase to be completed thoroughly before progressing to the next, ensuring clarity, reducing ambiguity, and minimizing the risk of missing key functional or design elements. This systematic progression supports a disciplined development process aligned with the defined goals and scope of the platform.

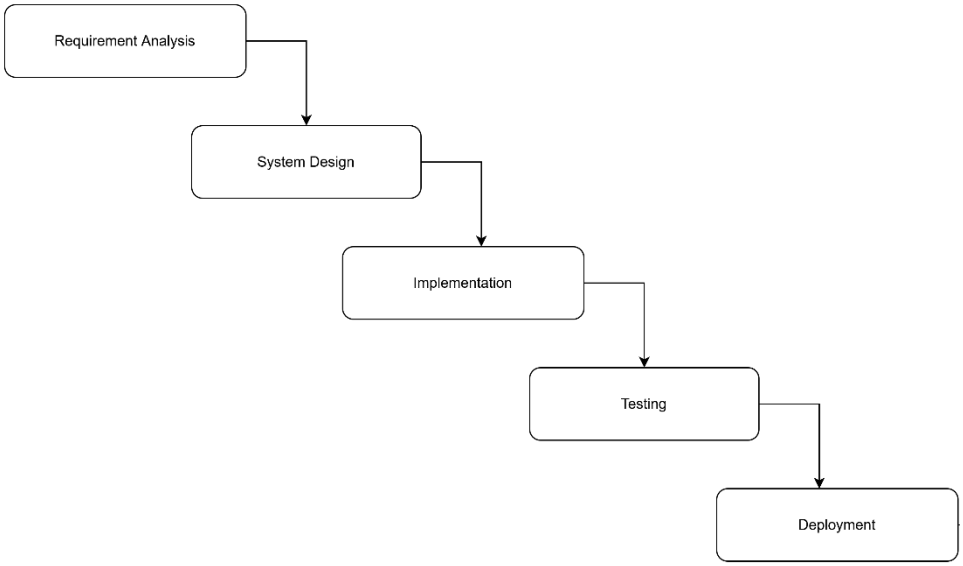


Figure 1.1: Waterfall Model of Interconnected Platform

Requirement Analysis Phase

An analysis of existing social media platforms reveals a strong reliance on engagement-driven algorithms that prioritize trending or sensational content, often leading to information overload and reduced focus on user interests. To address this, the Interconnected platform is structured around interest-based communities as the primary unit of interaction. Users can engage with one community at a time for a more focused experience, while features for community creation and moderation are supported by a central administrative role that ensures proper platform usage and system integrity.

System Design Phase

Based on the identified requirements, the system was structured using a three-tier architecture consisting of presentation, application, and data layers to support scalability, maintainability, and security. Entity Relationship Diagrams (ERDs) and Data Flow Diagrams (DFDs) were first developed to clearly define data relationships and system processes. Using these models as a reference, the database schema was then designed to support core entities such as users, communities, posts, and comments. As the design evolved, the schema was refined and updated to accommodate additional requirements, including reporting features and community privacy controls. Interface wireframes and mockups were also created to represent user interactions, with a strong emphasis on clarity, usability, and the single-community viewing approach. This design process provided a stable structural foundation for accurate and efficient system implementation.

Implementation Phase

The Implementation Phase involves translating the approved system design into a fully functional application. The frontend is developed using React with TypeScript, along with HTML5 and Tailwind CSS, to create a responsive, consistent, and user-friendly interface. Core interfaces are implemented for user authentication, community browsing, post creation, commenting, and administrative actions. The backend is developed using PHP to manage business logic, session handling, request processing, and communication with the database. A structured MySQL database is implemented based on the finalized schema. Modular development practices are followed to allow individual components to be built, tested, and refined before system integration. Version control is used throughout development to track changes and ensure consistency across the project.

Testing Phase

The Testing Phase verifies that the implemented system meets the defined functional and non-functional requirements. Unit testing is carried out on individual components to ensure that each function operates as expected. Integration testing is performed to validate interactions between the frontend, backend, and database, ensuring correct communication and data flow across system components. System testing is then conducted to evaluate the platform as a whole, focusing on user workflows, content management, moderation features, and administrative controls. Instead of formal user acceptance testing, feedback is obtained from project supervisors and instructors, and the system is refined based on their

suggestions. Identified bugs and usability issues are addressed through iterative fixes and retesting prior to final submission.

Deployment Phase

The Deployment Phase focuses on preparing the system for execution in a controlled local environment. Instead of deploying the platform to a live production server, the application is deployed locally using XAMPP, which provides the required Apache server and MySQL database services. The database is configured and initialized based on the designed schema, and server settings are adjusted to ensure proper application execution. Final checks are performed to verify system stability, functionality, and security within the local environment. This local deployment allows the complete system workflow to be demonstrated and tested effectively while providing a foundation for potential future deployment in a real-world environment.

1.6 Report Organization

This report is structured into five chapters, each describing a specific aspect of the design, development, and evaluation of the Interconnected platform.

Chapter 1: Introduction presents an overview of the Interconnected platform and the motivation for developing a community-focused social media system. It defines the problem statement, project objectives, scopes, and limitations. The chapter also explains the development methodology adopted for the project, including a detailed justification for using the Waterfall Model.

Chapter 2: Background Study and Literature Review discuss the background of social networking systems and highlights the need for a focused community-based platform. It includes a review of existing systems and related research, analyzing their strengths, limitations, and relevance to the proposed platform.

Chapter 3: System Analysis and Design describes the system analysis and design processes undertaken during development. It covers requirement analysis, feasibility study, system architecture, database schema design, and interface design. This chapter also includes various structural and process models such as Use Case Diagrams, Entity Relationship Diagrams (ERD), Data Flow Diagrams (Level 0 and Level 1), Gantt Chart, and Physical Data Flow Diagrams.

Chapter 4: Implementation and Testing details the implementation and testing phases of the Interconnected platform. It outlines the tools and technologies used for frontend and backend development, explains the implementation of major system modules, and describes the testing strategies employed, including unit testing, integration testing, system testing, and user acceptance testing.

Chapter 5: Conclusion and Future Recommendations summarizes the outcomes and achievements of the project. It reflects on the objectives, scopes, and limitations, discusses lessons learned during development, and provides recommendations for future enhancements and improvements to the platform.

The report concludes with a references section listing all sources consulted during the study and an appendix containing supplementary materials such as detailed diagrams, configurations, and supporting documentation.

CHAPTER 2
BACKGROUND STUDY AND LITERATURE REVIEW

2.1 Background Study

The evolution of social media has significantly transformed human interaction since the early days of the internet. Initial platforms such as bulletin board systems and chat rooms offered simple, user-controlled spaces for discussion. Early social networking sites, including Friendster and MySpace, emphasized personal connections and chronological content display, allowing users direct control over the information they consumed.

The emergence of large-scale platforms such as Facebook and Twitter marked a shift toward algorithmic content curation. Chronological feeds were replaced by engagement-driven algorithms designed to maximize likes, comments, and shares. While these systems increased user activity, they also introduced challenges including information overload, fragmented attention, and declining discourse quality.

Research shows that engagement-optimizing algorithms tend to amplify emotionally charged or divisive content due to higher interaction rates, creating feedback loops that prioritize visibility over accuracy. Excessive social media use has also been linked to negative mental health outcomes such as anxiety, depression, and reduced self-esteem, particularly among younger users.

The phenomenon of “doomscrolling” further illustrates the ethical concerns of algorithmic feeds, as platforms exploit negativity bias to encourage compulsive consumption despite negative emotional effects. In response, the digital wellness movement has promoted mindful technology use and influenced features such as screen-time tracking and notification controls, though these measures often fail to address the underlying engagement-driven design.

Community-based platforms such as Discord and Reddit present alternative models by organizing interaction around shared interests. However, Discord’s focus on real-time communication limits reflective discussion, while Reddit’s anonymity and voting mechanisms can reinforce echo chambers and suppress minority viewpoints.

Emerging “slow social” or “intentional social” design approaches advocate for meaningful engagement through chronological content, reduced interaction frequency, and quality-

focused contributions. Studies on online communities consistently emphasize the importance of clear guidelines, effective moderation, focused group size, and structured discussions. Research on attention and cognitive load further indicates that reduced context-switching improves comprehension and depth of engagement.

The Interconnected platform is developed in response to these findings. By eliminating algorithmic content curation, adopting single-community viewing, and emphasizing moderated, topic-focused interaction, the platform seeks to promote meaningful engagement aligned with principles of digital wellness and intentional technology use.

2.2 Literature Review

Recent research on social media platforms highlights growing concerns about engagement-driven design and its impact on user wellbeing, attention, and discourse quality. Studies show that algorithmic content curation systems, optimized primarily for engagement metrics, reduce user agency and often amplify emotionally charged or polarizing content. Chan et al. conducted an empirical audit of Reddit's algorithmic ranking and demonstrated that engagement-based visibility strongly shapes user exposure while remaining largely opaque to users, reinforcing attention loops rather than intentional consumption [1]. Such findings support critiques of algorithmic feeds identified in the background study and motivate alternative feed designs that prioritize transparency and user control.

Research in human–computer interaction has increasingly explored intentional or user-directed social media models as a response to these challenges. El Malki et al. proposed the BONSAI system, which replaces passive algorithmic inference with explicit user-defined intentions to generate social feeds [2]. Their findings indicate that aligning content delivery with stated user goals improves perceived control and reduces mindless consumption. This approach directly informs Interconnected's design choice to eliminate algorithmic ranking in favor of user-controlled, chronological, and community-scoped content organization.

Digital wellbeing and attention research further reinforces the need for structural design changes rather than surface-level controls. Roffarello and De Russis, in a systematic review, found that conventional self-control tools such as timers and blockers have limited long-term effectiveness when underlying platform incentives remain unchanged [3]. Complementing this, Davis et al. demonstrated that brief moments of reflection and intention-setting before social media use significantly improve users' sense of autonomy and reduce compulsive engagement [4]. These findings support Interconnected's single-

community viewing model and intentional interaction flow, which aim to reduce context-switching and promote focused participation.

Community-focused governance has also been identified as a critical factor in sustaining healthy online environments. Jhaver et al. examined multi-level governance structures and showed that platforms enabling community-level moderation and rule-setting foster stronger norms, accountability, and discourse quality compared to centralized moderation alone [5]. This research underpins Interconnected's emphasis on topic-focused communities with active, localized moderation rather than global engagement optimization.

Behavioral design literature further explains how platform environments shape user behavior through subtle cues rather than explicit enforcement. Clear's Atomic Habits emphasizes that habits are largely formed through environmental design, where small structural cues guide behavior automatically rather than through conscious effort [6]. This concept aligns with the "invisible hand" perspective adopted in Interconnected, where interface constraints and defaults nudge users toward intentional, meaningful engagement without relying on persuasive algorithms.

Overall, the reviewed literature provides a focused theoretical foundation for the Interconnected platform. By addressing algorithmic amplification, attention fragmentation, and governance limitations identified in recent research, the platform adopts evidence-based design principles that emphasize user intent, community moderation, and digital wellbeing.

CHAPTER 3
SYSTEM ANALYSIS AND DESIGN

3.1 System Analysis

For the Interconnected platform, system analysis involved studying the needs of different users such as community members, group managers, and administrators to understand their expectations and challenges with existing social media platforms. It was observed that most platforms are heavily algorithm-driven, often prioritizing trending content over meaningful or creative interactions. Based on these insights, the analysis identified the key requirements for the platform, including focused community viewing, content creation and moderation tools, and administrative controls. This phase provided a clear understanding of functional requirements, operational constraints, and user interactions, forming the foundation for subsequent system design, development, testing, and deployment.

3.1.1 Requirement Analysis

The requirement analysis for the Interconnected platform focused on identifying the features and capabilities needed to support different user roles, including general users, group managers, and system administrators. Key functionalities such as content creation, commenting, liking, reporting, and moderation were identified as essential for meaningful community interaction. The analysis drew on observations of existing social media platforms, highlighting issues such as algorithm-driven feeds, distraction, and lack of focused community engagement. Based on these insights, the requirements were documented to ensure clear expectations for each role, guiding the development of features like single-community viewing, community management, and administrative oversight. This process provided a foundation for the design and implementation phases, ensuring that the platform would be user-centered, organized, and scalable.

i. Functional Requirements

The Interconnected platform is a web-based application designed to facilitate structured interaction among users within topic-focused communities. The system provides functionality that allows users to create accounts, authenticate securely, join or request access to communities, and participate in discussions through posts and comments. Users can view content within a single community at a time to reduce information overload and encourage focused engagement. Community moderators are provided with tools to manage

membership requests, enforce community guidelines, moderate content, and handle reports submitted by users. Administrative users have access to platform-wide management features, including user management, community oversight, and moderation support. The system also supports content reporting, notification mechanisms, and role-based access control to ensure a secure and well-regulated environment.

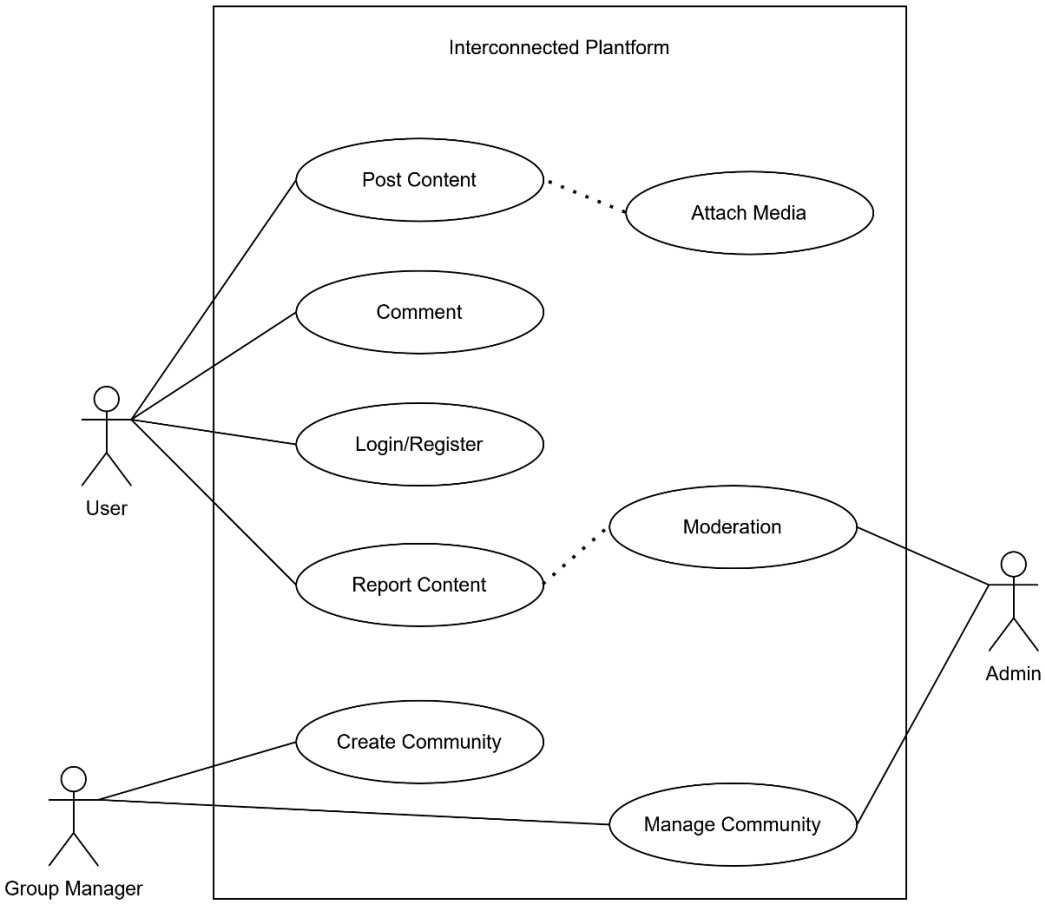


Figure 3.1: Use Case Diagram of Interconnected Platform

Admin:

The administrator must be authenticated before accessing the system. Once logged in, the admin can access the administrative dashboard to manage users, oversee communities, review reported content, and enforce platform-wide policies.

Group Manager (Community Moderator):

The group manager must be authenticated to access community management features. Their primary responsibility is to approve or reject membership requests for private

communities they manage. Group managers do not have permission to moderate posts or comments.

User:

Registered users can access the platform after logging in. Users are able to create posts, attach media, comment on community content, receive notifications, and report inappropriate content. They can participate in communities of interest but cannot perform moderation or administrative actions without the appropriate privileges.

ii Non-Functional Requirements

Performance and Responsiveness

- The website should load quickly and respond promptly to user actions.
- It should handle many users at the same time without slowing down.
  Accessibility
- Users should be able to access the system from anywhere with a stable internet connection.

Usability

- The website should be simple and easy to use.
- The interface should be intuitive and work on desktop, tablet, and mobile devices.
  Security
- User data, including passwords and payment information, must be protected.
- Communication should be secure (HTTPS), and user inputs must be validated.

Reliability and Availability

- The system should be available 24/7 with minimal downtime.
- Errors should be handled properly, and data should be backed up regularly.

Maintainability

- The system should be easy to update and maintain.
- Code should be organized and documented for future improvements.

3.1.2 Feasibility Study

A feasibility study evaluates whether the proposed system can be successfully developed and operated, considering available resources, technology, time, and operational factors.

For Interconnected, the study examines technical, operational, economic, and schedule feasibility to determine practicality and identify any risks.

a. Economic Feasibility

The system uses free and open-source technologies such as React with TypeScript, PHP, MySQL, and XAMPP, so there are no software licensing costs. Development can be done on existing computers, and internet and electricity costs are minimal. Overall, the project is financially feasible for a college setting, with no significant expenses.

b. Technical Feasibility

The required technologies are mature, well-documented, and suitable for building the platform:

Frontend Technologies: The interface will be built using React with TypeScript, HTML, and CSS. TypeScript ensures fewer errors due to its strong typing, and React supports dynamic, responsive interfaces. Features such as user profiles, posts, comments, and single-community viewing are fully achievable with this stack.

Backend Technologies: PHP handles server-side operations including authentication, session management, and database interactions. PHP is widely used and well-documented.

Database Technology: MySQL is a reliable relational database suitable for storing users, communities, posts, and relationships. It provides indexing, transactions, and data integrity. Administration tools are available for easy management.

Development Environment: XAMPP provides a complete local environment including Apache, MySQL, and PHP, allowing full development and testing on a local machine.

The technical feasibility confirms that all required resources are available and there are no significant technical barriers to completing the project.

c. Operational Feasibility

The system is designed to be intuitive and easy to use for both students and administrators:

User Perspective: Users will find the platform accessible and user-friendly. It follows familiar social media patterns (profiles, posts, comments) with enhancements like single-community viewing. Minimal learning is required for students already familiar with online communities.

Administrator Perspective: The admin dashboard and moderation tools provide full control over communities. The interface is simple and does not require extensive training.

Maintenance Perspective: The modular design allows easy updates and maintenance. Documentation created during development supports future maintenance.

Overall, the system is practical, usable, and maintainable in a college project setting.

d. Schedule Feasibility

The project can be completed within the available timeframe using a Waterfall development model, which organizes tasks sequentially. Each phase (analysis, design, implementation, testing, deployment) follows a clear order, reducing confusion and ensuring systematic progress.

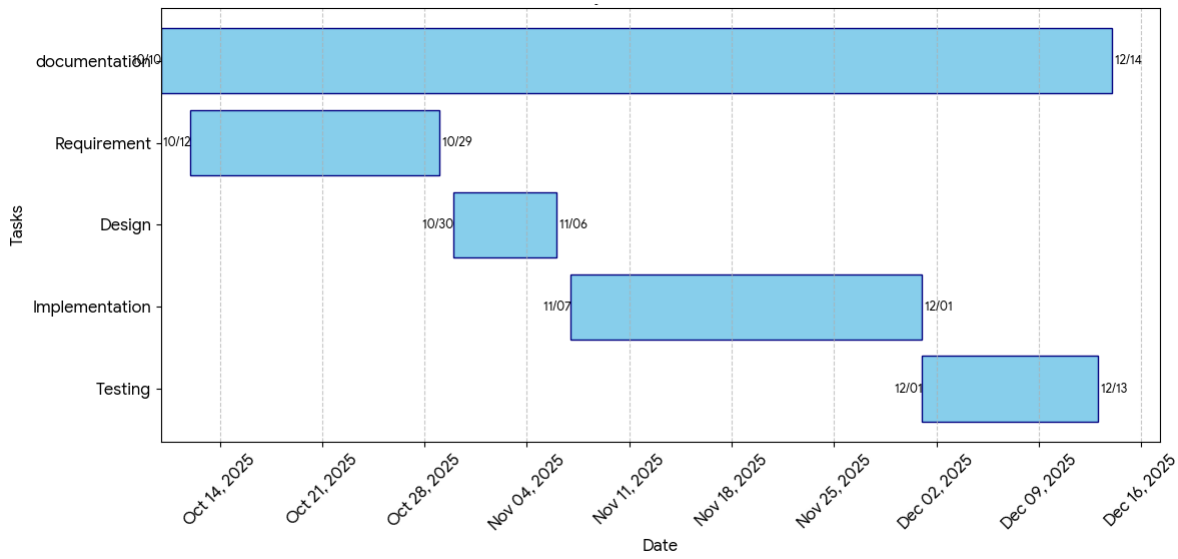


Figure 3.2 Gantt Chart of interconnected Platform

Table: 3.1 Gantt Table of interconnected Platform

Task	Start date	Duration	End date
Documentation	10/10/2025	66	12/14/2025
Requirement	10/12/2025	17	10/29/2025
Design	10/30/2025	7	11/6/2025
Implementation	11/7/2025	24	12/1/2025
Testing	12/1/2025	12	12/13/2025

3.1.3 Data Modeling

Data modeling is the process of creating a conceptual representation of the data structures, entities, relationships, and constraints that will be stored in a system's database. For Interconnected, data modeling uses Entity Relationship Diagrams (ERD) to visualize the database schema and illustrate the relationships between different data entities. ERDs provide a clear structure for developers and stakeholders to understand how data is organized, maintained, and interconnected.

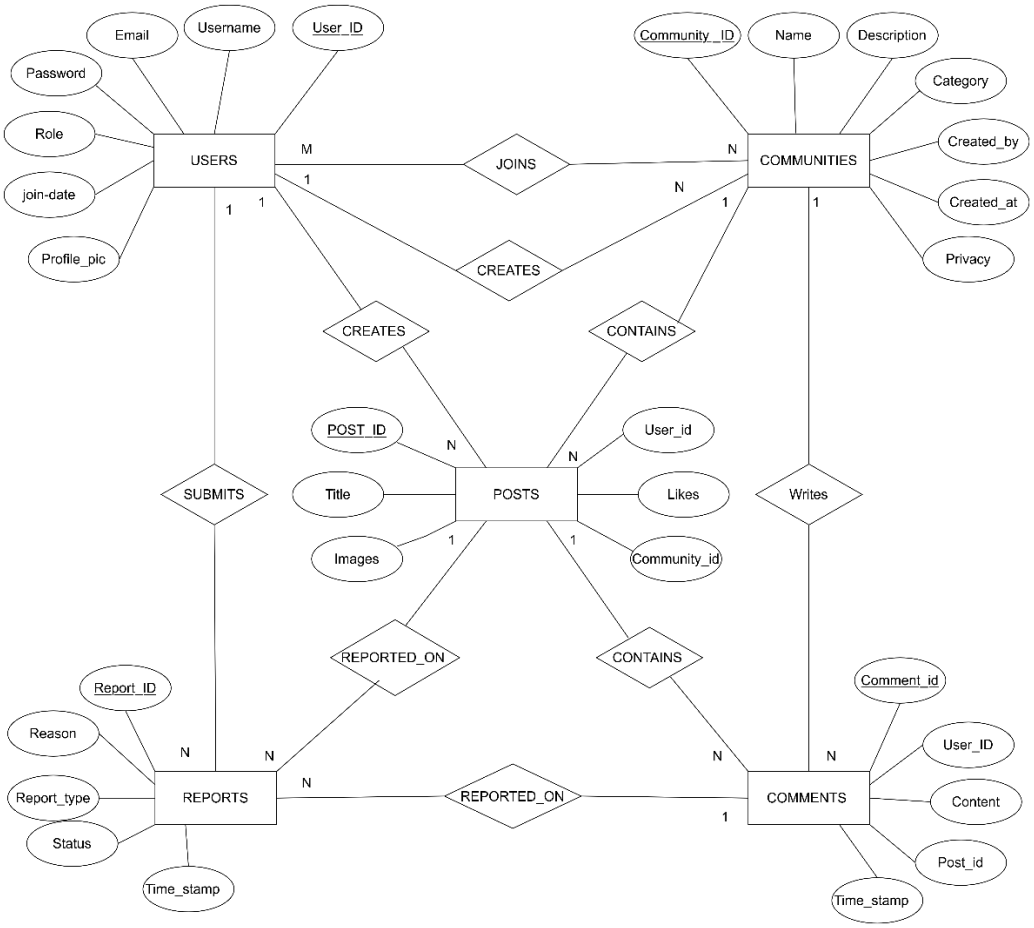


Figure 3.3: Entity Relationship Diagram of Interconnected Platform

Figure 3.3: The Entity Relationship Diagram for Interconnected provides a visual representation of the relationships and interactions between the primary entities such as User, Community, Post, Comment, Membership, and Report. It shows how users can register, log in, join communities, create posts, and comment on content, highlighting the connections between these entities and their attributes. The diagram also illustrates the roles and functionalities available to administrators and moderators, including managing communities, monitoring content, and handling reports. By visually mapping the database schema and the relationships between these components, the ER diagram serves as a clear tool for understanding the underlying structure, data flow, and functionality of the Interconnected platform's database.

3.1.4 Process Modeling

Process modeling involves graphically representing the processes that capture, manipulate, store, and distribute data between a system and its environment. For Interconnected, Data Flow Diagrams (DFDs) are used to illustrate how data moves through the system, showing processes, data stores, external entities, and the flow of information. Multiple levels of DFDs provide different views of the system, from a high-level overview to more detailed process specifications.

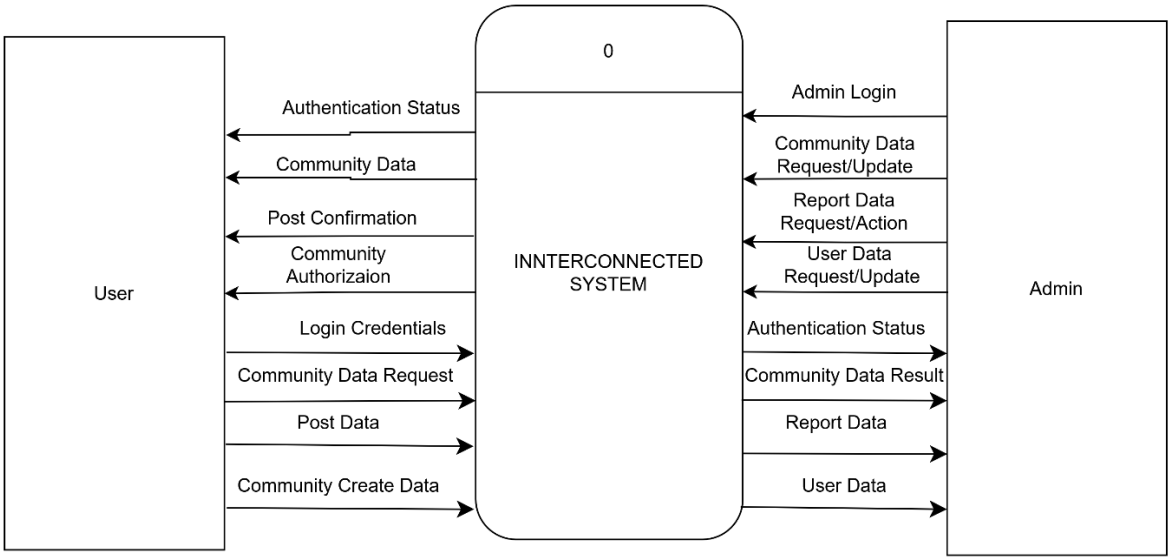


Figure 3.4: Context Level Diagram of Interconnected Platform

The Context Level Diagram provides the highest-level view of the Interconnected system, showing it as a single process interacting with external entities:

- User: Registered users who can log in, browse communities, view posts, create content, and perform other platform actions.
- Administrator: Platform administrators responsible for managing users, communities, and reported content.

This diagram establishes the system boundary, highlighting interactions between external entities and the Interconnected platform.

Level 1 DFD – User Processes:

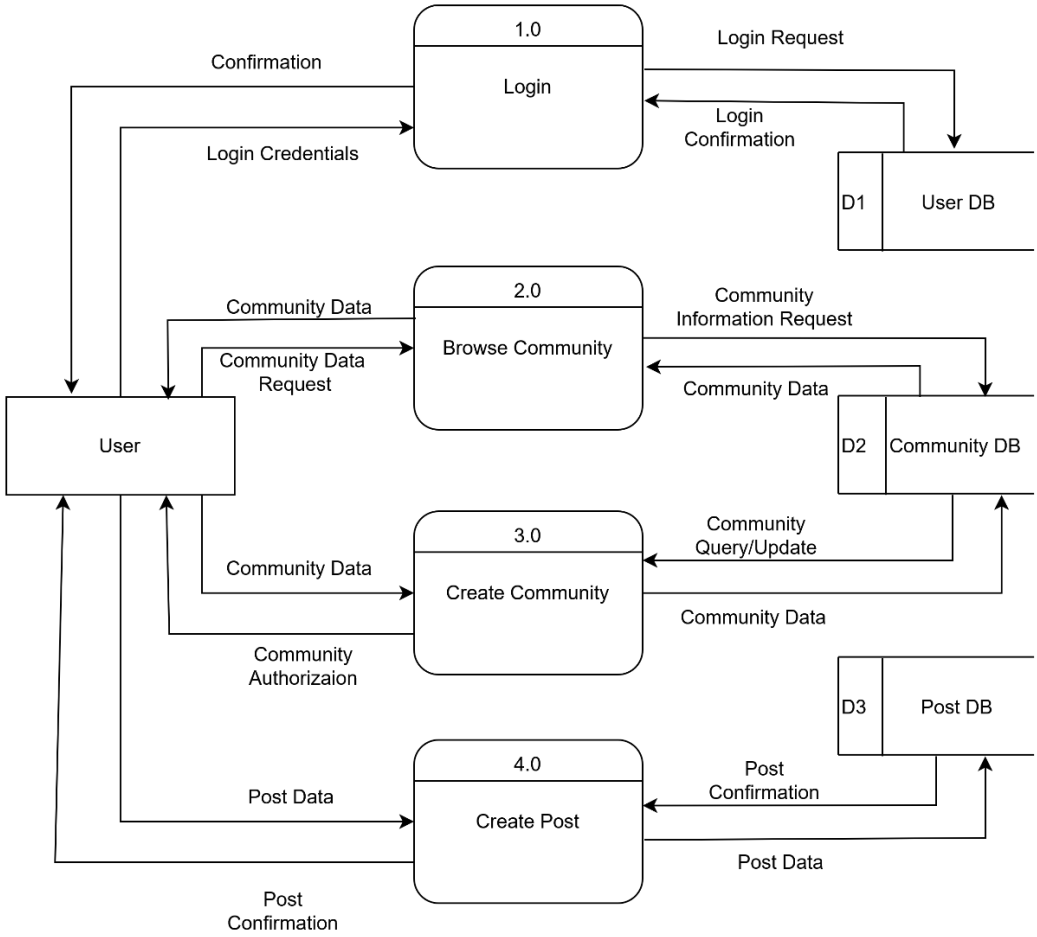


Figure 3.5: User Level 1 Data Flow Diagram of Interconnected Platform

The Level 1 DFD for users decomposes the system into major functional processes:

- User Authentication: Users log in and receive authentication responses.
- Community Discovery & Joining: Users can browse communities, search by category, and join communities.
- Community Creation & Management: Users can create new communities. Upon creation, they are assigned the role of community manager, granting them moderation privileges within their community.
- Content Viewing & Creation: Users can view posts and comments or create new posts/comments within communities they are members of.
- Content Interaction: Users can report inappropriate content.

The diagram shows how data flows between users and the system's core functions while interacting with relevant data stores, including User, Community, Membership, Post, and Comment data stores.

Level 1 DFD – Administrator Processes:

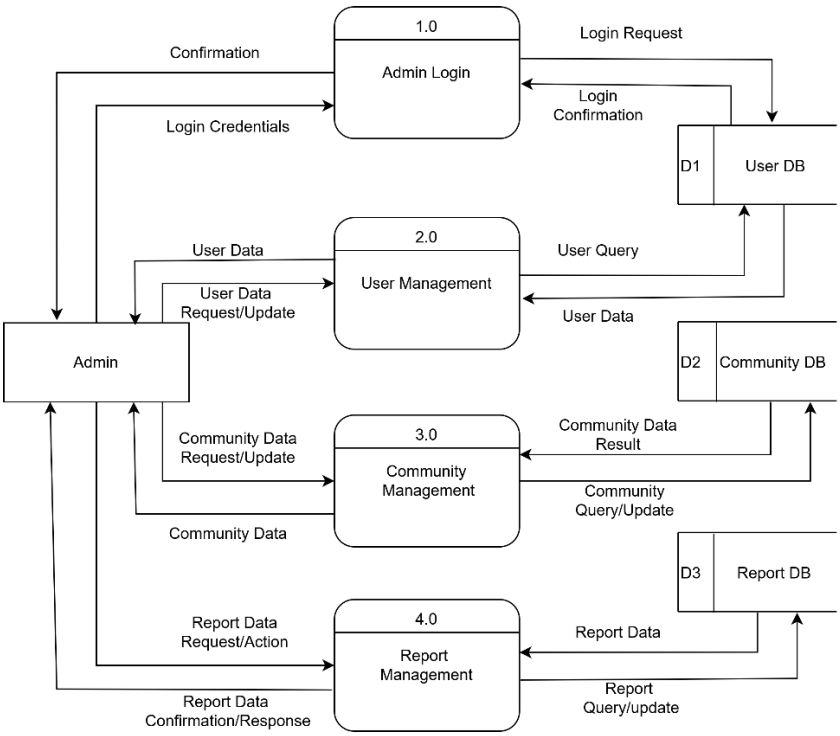


Figure 3.6: Admin Level 1 Data Flow Diagram of Interconnected Platform

The Level 1 DFD for administrators shows the main management and moderation processes:

- Admin Authentication: Administrators log in and receive authentication responses.
- User Management: Administrators view, suspend, or ban users.
- Community Management: Administrators manage communities and assign moderators.
- Content Moderation: Administrators review reported posts and comments.
- Analytics & Monitoring: Administrators generate reports and monitor platform activity.
- System Configuration: Administrators update platform settings and policies.

This diagram illustrates how administrators interact with the system independently of regular users while accessing the same underlying data stores.

3.2 System Design

System design is the process of defining a system's components, including modules, architecture, interfaces, and data, based on the requirements identified during analysis. Design serves as the bridge between system analysis and implementation. In the design phase of Interconnected, several design artifacts were created, including system architecture, database schema design, UI interface and structure diagrams, and physical DFDs.

3.2.1 System Architecture

The system architecture of Interconnected is divided into three main layers: presentation layer, application layer, and database layer.

Presentation Layer:

The presentation layer handles all interactions with users, receiving their requests and providing responses. Users access the platform via web browsers, where the interface displays content and functionalities. The frontend is built using React.js for dynamic, responsive, and interactive interfaces, alongside HTML5 for structure and CSS3 for styling. These technologies ensure smooth user experience for activities such as registering, logging in, browsing communities, creating posts, and managing content.

Application Layer:

The application layer, also called the logic layer, processes requests from the presentation layer. It implements the business logic of the platform, handling authentication, community management, content creation, reporting, and moderation workflows. PHP is used for server-side development, providing secure processing of user requests, session management, and interaction with the database. This layer ensures that user actions are validated, processed, and returned with the appropriate responses.

Database Layer:

The database layer stores all persistent data of the Interconnected platform. It manages users, communities, posts, comments, memberships, and reports. MySQL is used as the relational database management system, ensuring data integrity, support for transactions, indexing for performance, and structured storage. This layer allows the application to

retrieve and update data efficiently, supporting platform functionalities such as community discovery, content viewing, user management, and reporting.

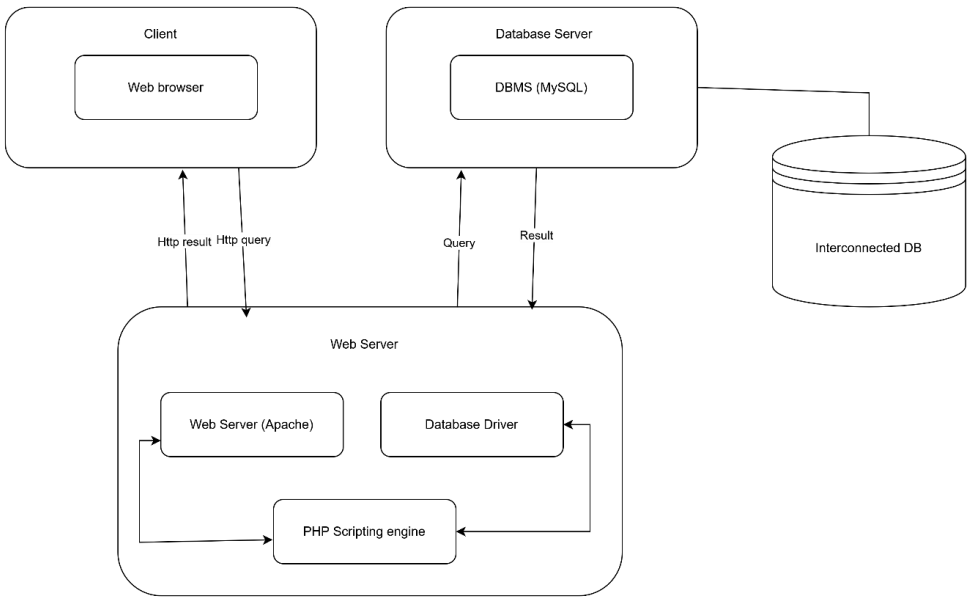


Figure 3.7: System Architecture of Interconnected Platform

3.2.2 Database Schema Design

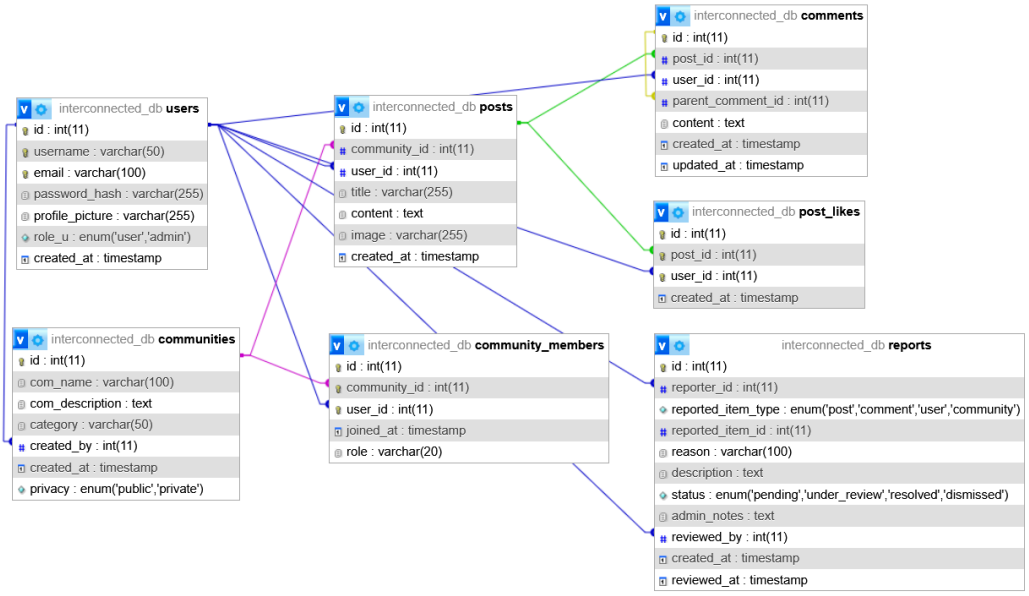


Figure 3.8: Database Schema Design of Interconnected Platform

The database schema for Interconnected provides a structured representation of the platform's data and the relationships between different entities. The relational database

design organizes information across several interconnected tables, including Users, Communities, Community Members, Posts, Post Likes, Comments, and Reports.

- Users Table: Stores information about registered users, including username, email, password hash, profile picture, role, and account creation timestamp. Indexes on email and username optimize search and authentication queries.
- Communities Table: Manages information about communities, including name, description, category, creator, creation date, and privacy level. Each community references a creator from the Users table.
- Community Members Table: Represents the many-to-many relationship between users and communities, tracking membership, roles (member, moderator), and join date. Unique constraints ensure a user cannot join the same community twice.
- Posts Table: Contains posts created within communities, including title, content, optional images, creation timestamp, and references to the author and community.
- Post Likes Table: Tracks likes on posts, ensuring each user can like a post only once, with foreign key relationships to posts and users.
- Comments Table: Stores comments and replies on posts, supporting hierarchical relationships through a parent comment reference. Includes timestamps for creation and updates.
- Reports Table: Captures user-reported content requiring moderation. It stores the reporter, type of reported item (post, comment, user, or community), reason, description, status, assigned moderator, and timestamps.

Foreign keys enforce referential integrity between tables, ensuring consistent relationships between users, communities, posts, comments, and reports. Indexes on frequently queried fields enhance performance, enabling efficient data retrieval for platform functionalities such as user management, community interaction, content moderation, and reporting.

Overall, this schema supports the core functionalities of Interconnected, facilitating scalable, secure, and organized storage of all platform data.

3.2.3 Interface Design (UI Interface/ Interface Structure Diagrams)

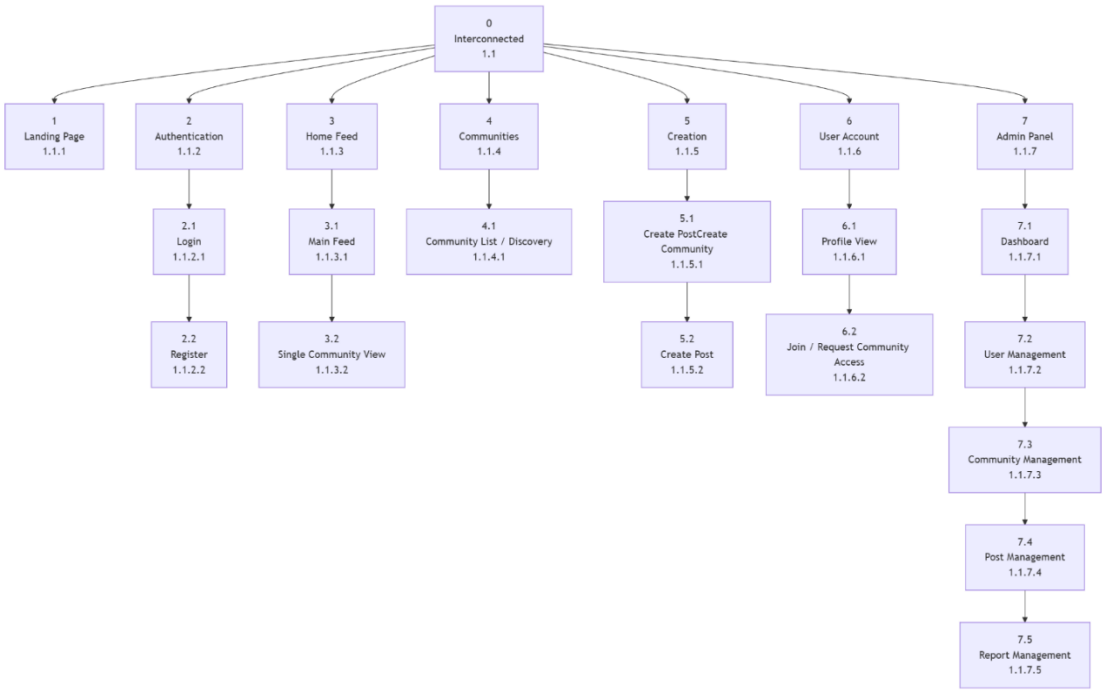


Figure 3.9: Interface Design of Interconnected Platform

The site map for the Interconnected social platform outlines a hierarchical navigation structure designed to support community interaction and content sharing. The interface is divided into public-facing pages and a secured authenticated environment, with separate navigation paths for regular users and administrators.

The top-level sections include the Landing Page, Authentication pages (Login/Register), and the Main Application Dashboard (Home). These sections provide initial access to the platform and guide users into the system.

After successful authentication, the Home section acts as the central hub of the platform. From this hub, users can navigate to key functional areas such as the Feed, which displays community-based posts and discussions, and the Community Hub, where users can discover communities, view individual community pages, and access member and post listings. A dedicated Create Community interface allows users to establish new communities and automatically become community managers.

The interface also includes Content Creation features, enabling users to create posts within communities. The User Account section provides access to personal profile information and activity history.

For users with administrative privileges, the Admin Panel offers a separate, role-restricted interface. It includes a Dashboard Overview displaying system statistics, along with modules for User Management, Community Management, Post Management, and Report Management. These interfaces support moderation, system monitoring, and administrative control.

Overall, the interface design demonstrates clear navigation paths and logical separation of functionalities. User-focused features prioritize engagement and community participation, while administrative tools are isolated to ensure secure and efficient platform management.

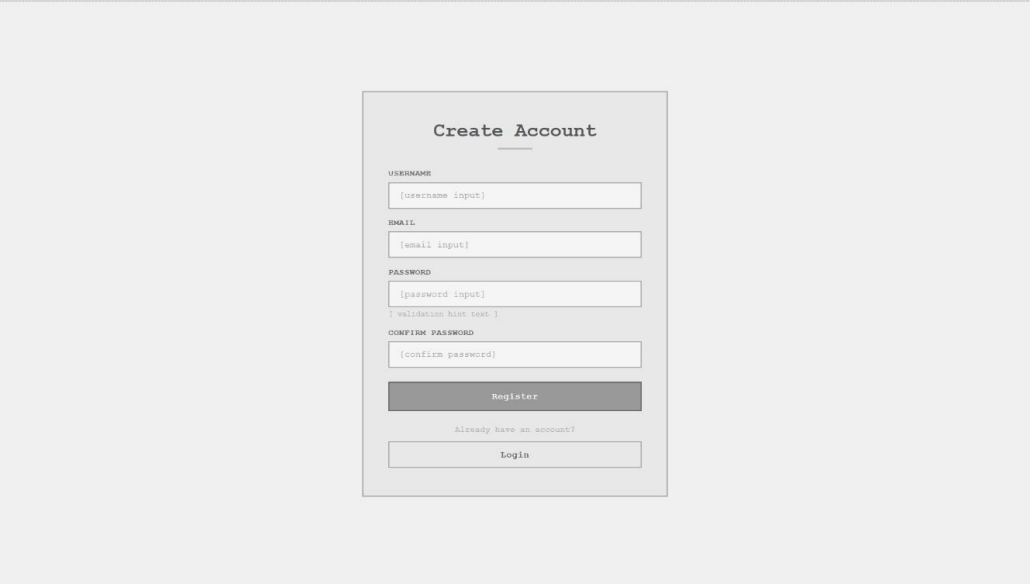


Figure 3.10: Low Fidelity Design Registration Page

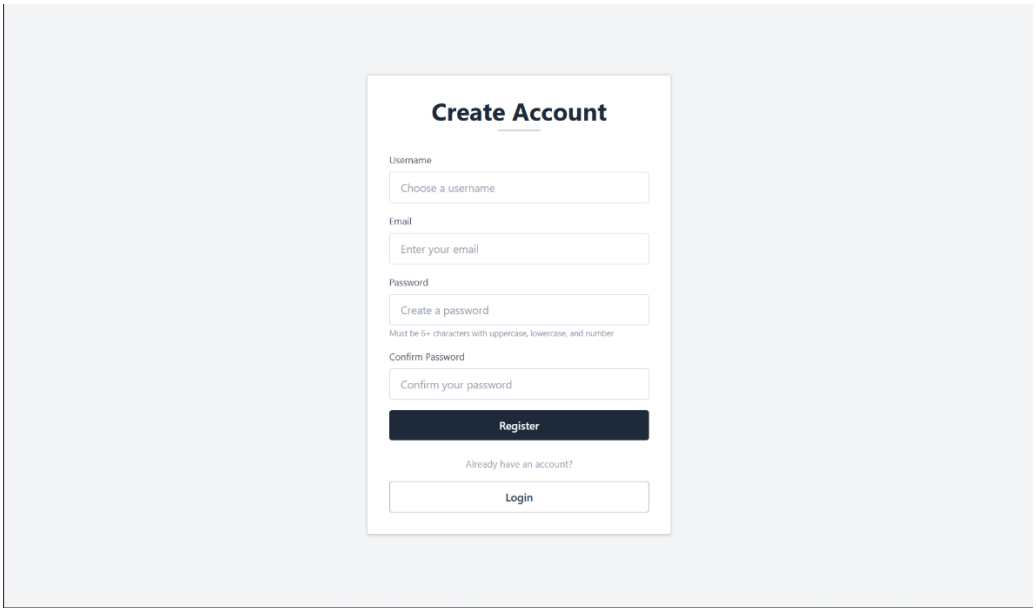


Figure 3.11: High Fidelity Design Registration Page

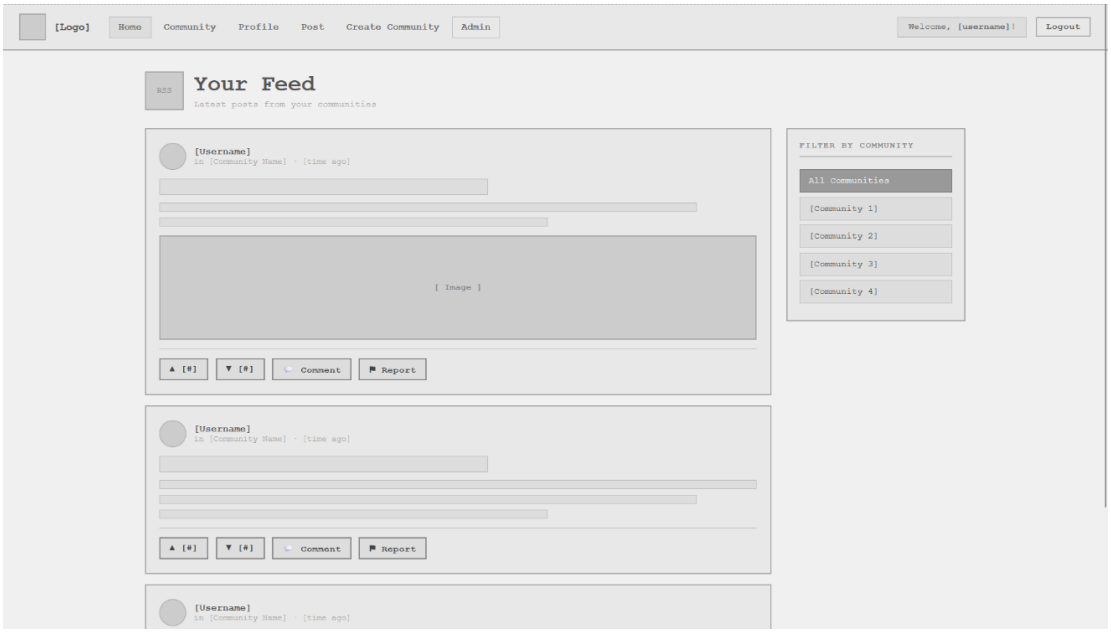


Figure 3.12: Low Fidelity Design Home Page

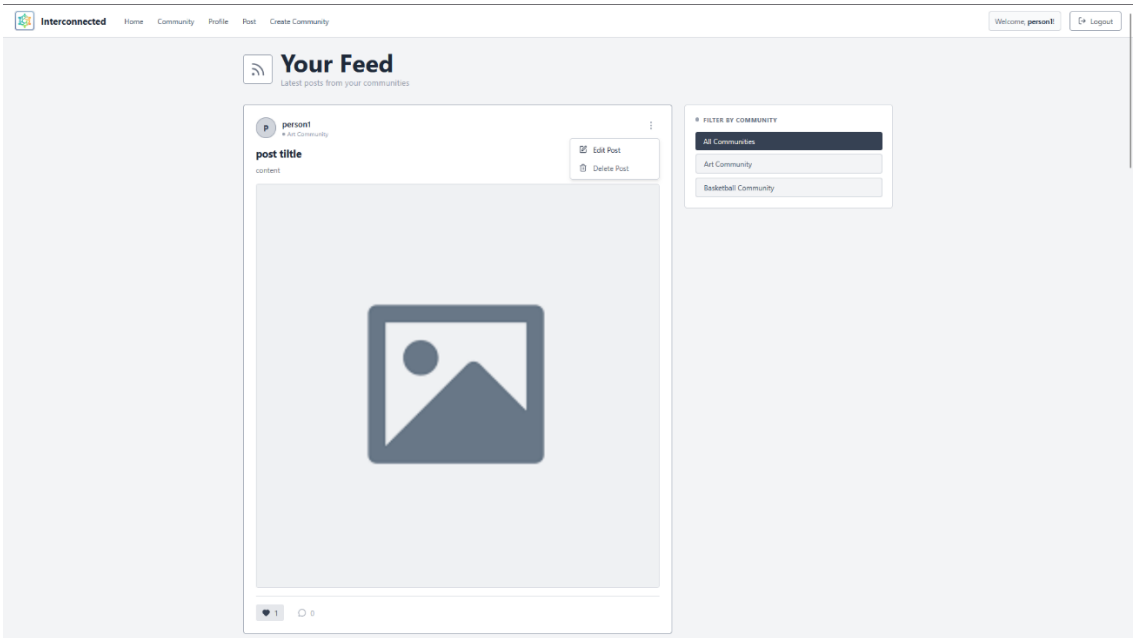


Figure 3.13: High Fidelity Design Home Page

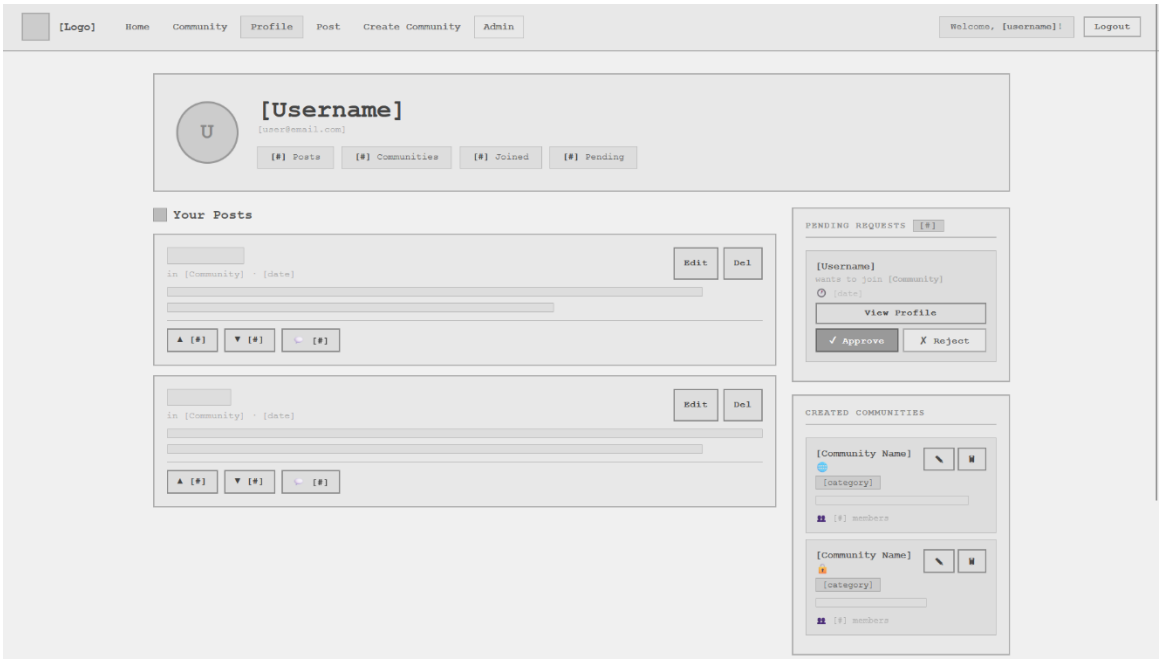


Figure 3.14: Low Fidelity Design Profile Page

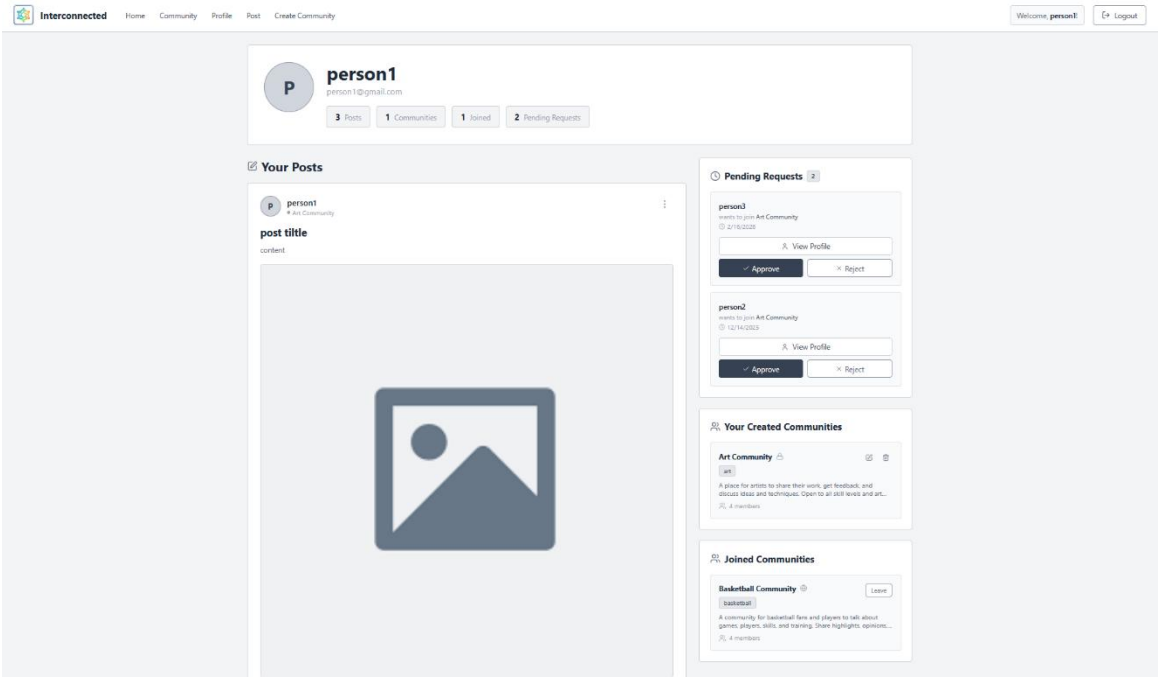


Figure 3.15: High Fidelity Design Profile Page

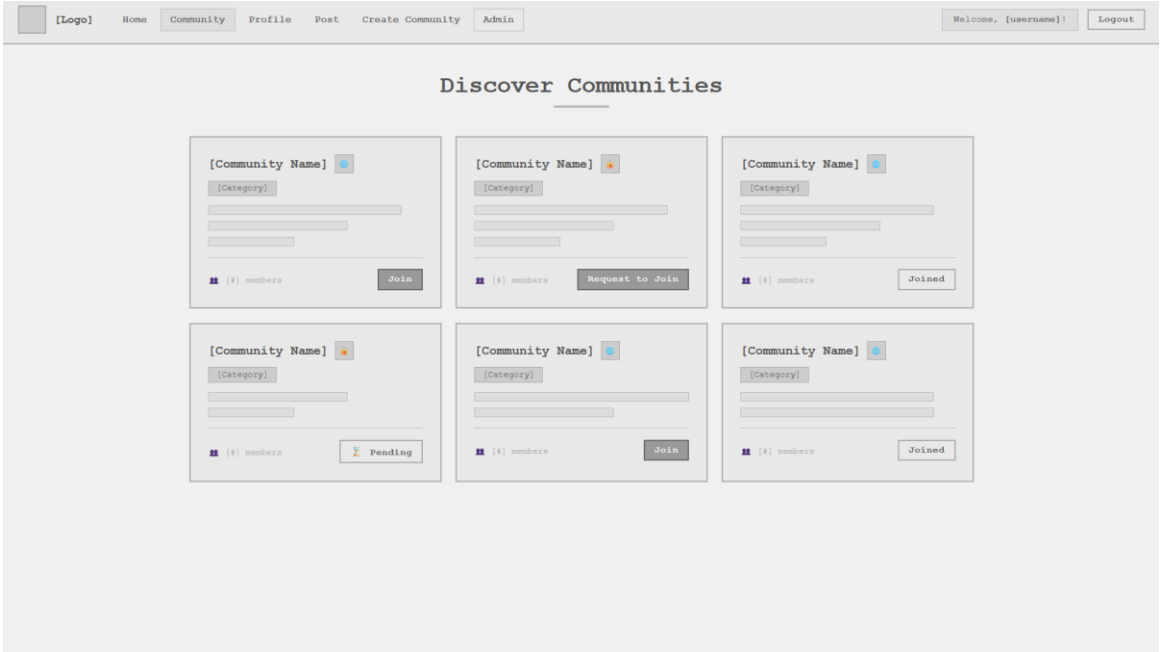


Figure 3.16: Low Fidelity Design Communities Page

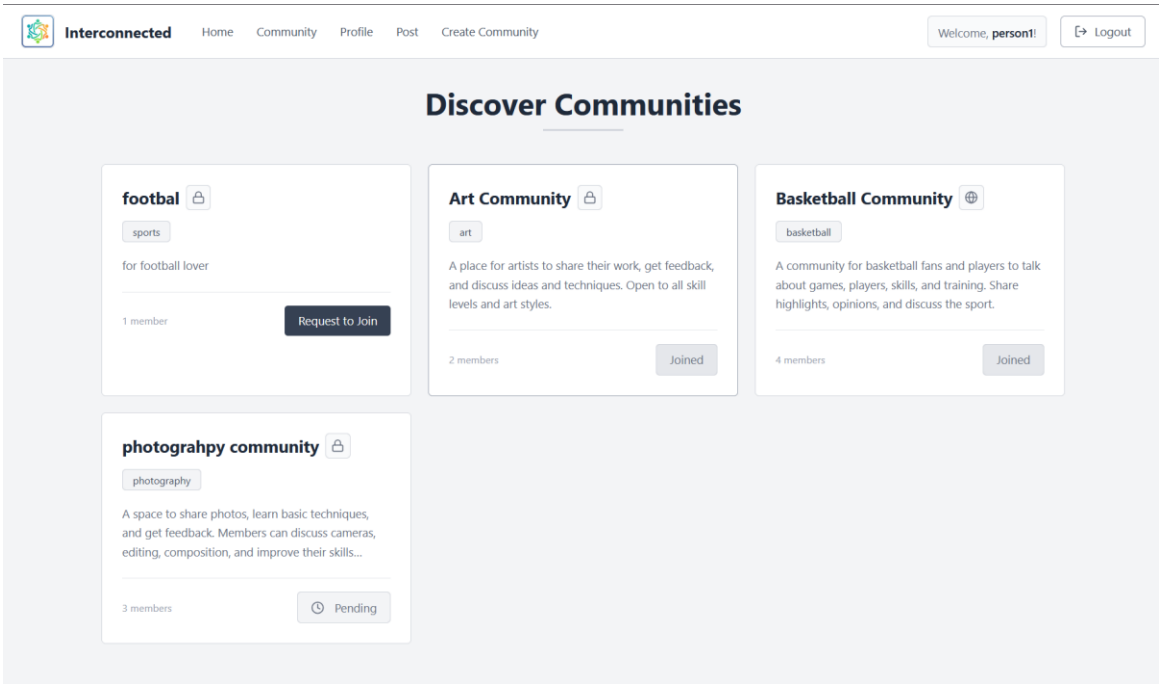


Figure 3.17: High Fidelity Design Communities Page

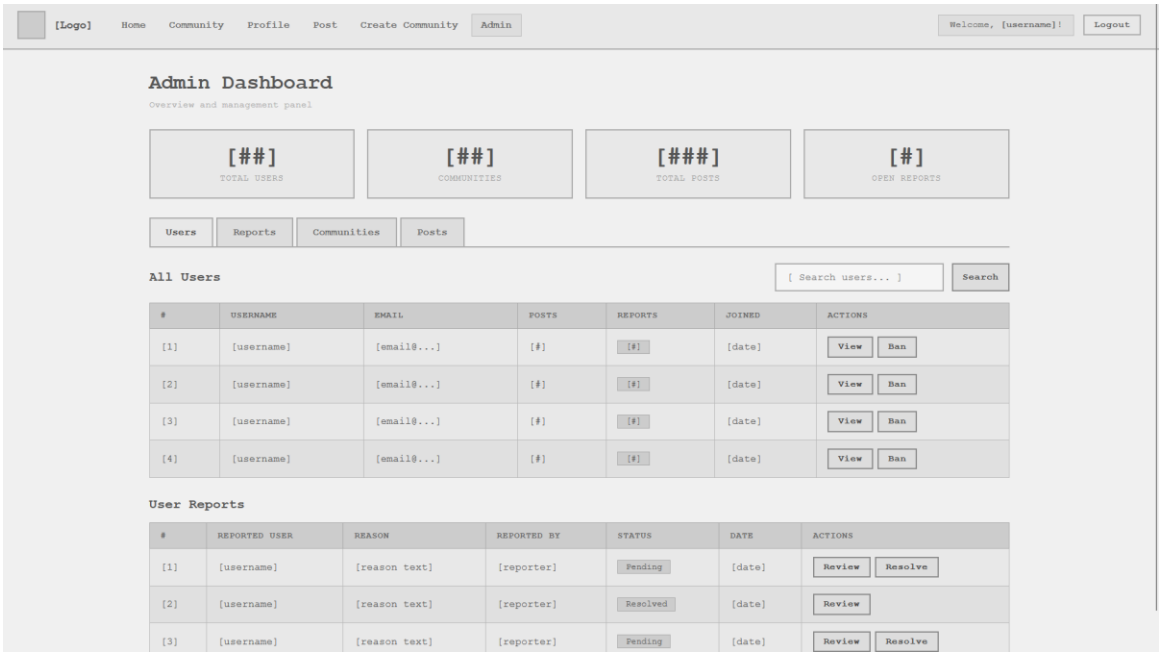


Figure 3.18: Low Fidelity Design Admin Page

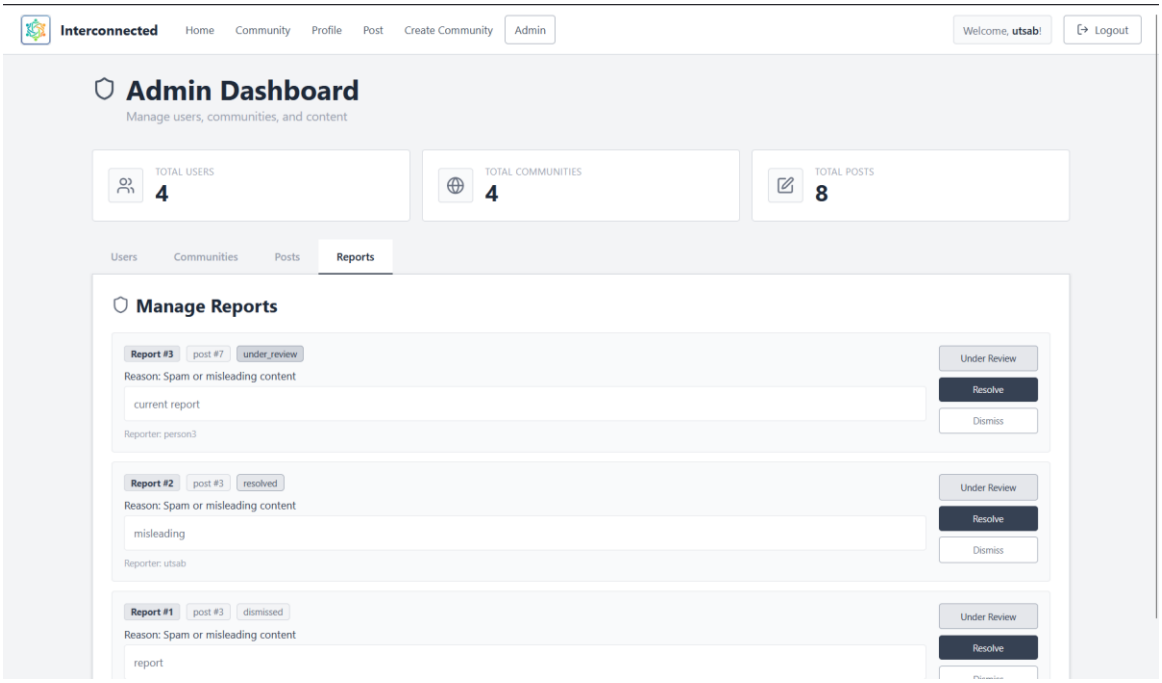


Figure 3.19: Low Fidelity Design Admin Page

3.2.4 Physical DFD

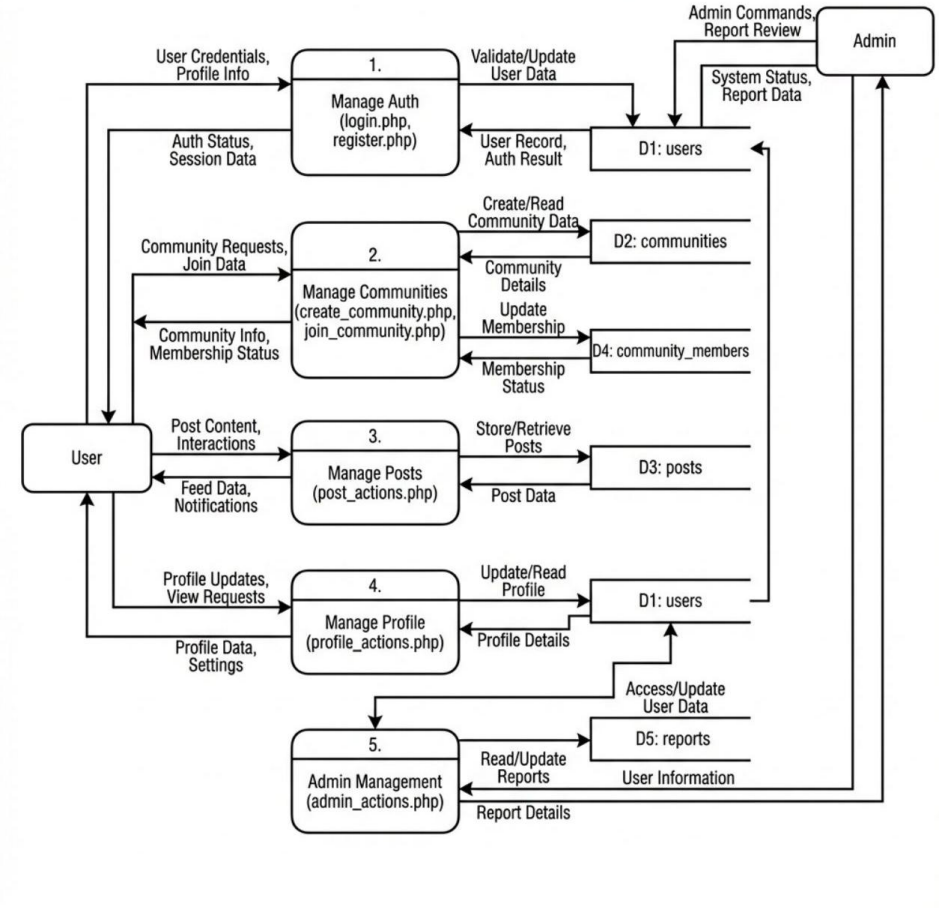


Figure 3.20: Physical Diagram of Interconnected Platform

Unlike logical DFDs, which focus on the system's functionality, physical DFDs provide a detailed view of the system's implementation, showing the actual components and devices involved in the data flow.

CHAPTER 4
IMPLEMENTATION AND TESTING

4.1 Implementation

Implementation is the phase where the designed system is actually built, translating specifications and designs into working code. This chapter details the tools and technologies used, describes the implementation of major system modules, and documents the testing process that verified system functionality.

4.1.1 Tools Used

The Interconnected platform was implemented using a carefully selected set of tools and technologies. This section describes each tool, its purpose, and how it was used in the project.

Table 4.1: Tool used

Category	Tool	Purpose
Frontend	HTML5	Structure and semantic layout of web pages
Frontend	CSS3	Styling, layout, and responsive design
Frontend	JavaScript (ES6+)	Client-side interactivity and logic
Frontend	React.js	Building reusable UI components and routing
Backend	PHP	Server-side logic and API handling
Backend	MySQL	Database management and data storage
Development Environment	XAMPP	Local server environment (Apache, PHP, MySQL)
Development Environment	Git	Version control and source code management

4.1.2 Implementation of Modules

4.1.2.1 User Authentication Module:

Handles registration, login, and session management. Registration validates input, ensures unique usernames, hashes passwords, and creates a session. Login verifies credentials, generates session tokens, and returns user data. Sessions are secured with appropriate flags, minimal data storage, timeout, and logout functionality.

```php
$password = isset($data["password"]) ? $data["password"] : "";

if (empty($email) || empty($password)) {
    echo json_encode([
        "success" => false,
        "message" => "Email and password are required"
    ]);
    exit();
}

$query = "SELECT id, username, email, password_hash, role_u FROM users WHERE email = ?";
$stmt = mysqli_prepare($conn, $query);
mysqli_stmt_bind_param($stmt, "s", $email);
mysqli_stmt_execute($stmt);
$result = mysqli_stmt_get_result($stmt);
$row = mysqli_fetch_assoc($result);

if ($row && password_verify($password, $row["password_hash"])) {
    $_SESSION["id"] = $row["id"];
    $_SESSION["username"] = $row["username"];
    $_SESSION["email"] = $row["email"];
    $_SESSION["role_u"] = $row["role_u"];

    echo json_encode([
        "success" => true,
        "message" => "Login successful",
        "user" => [
            "id" => $row["id"],
            "username" => $row["username"],
            "role_u" => $row["role_u"]
        ]
    ]);
    exit();
}

echo json_encode([
    "success" => false,
    "message" => "Invalid email or password"
]);

mysqli_close($conn);
exit();
```

4.1.2.2 Community Management Module:

Manages creation, discovery, membership, and viewing of communities. Creation ensures unique names, assigns creator as moderator, and initializes membership. Discovery supports pagination, filtering, and search. Membership functions allow users to join/leave communities with updates to member counts. Community views include details, guidelines, and paginated posts with metadata.

```
78  else if ($action === 'approve_request' && $_SERVER['REQUEST_METHOD'] === 'POST') {
79      $data = json_decode(file_get_contents("php://input"), true);
80      $request_id = intval($data["request_id"]);
81
82      // Verify user is the creator of the community
83      $verify_sql = "SELECT cm.community_id, cm.user_id
84                     FROM community_members cm
85                     INNER JOIN communities c ON cm.community_id = c.id
86                     WHERE cm.id = ? AND c.created_by = ?";
87      $verify_stmt = mysqli_prepare($conn, $verify_sql);
88      mysqli_stmt_bind_param($verify_stmt, "ii", $request_id, $user_id);
89      mysqli_stmt_execute($verify_stmt);
90      $verify_result = mysqli_stmt_get_result($verify_stmt);
91
92      if (mysqli_num_rows($verify_result) === 0) {
93          echo json_encode([
94              "status" => "error",
95              "message" => "You don't have permission to approve this request"
96          ]);
97          exit();
98      }
99
100     // Update role to member
101     $update_sql = "UPDATE community_members SET role = 'member' WHERE id = ?";
102     $update_stmt = mysqli_prepare($conn, $update_sql);
103     mysqli_stmt_bind_param($update_stmt, "i", $request_id);
104
105     if (mysqli_stmt_execute($update_stmt)) {
106         echo json_encode([
107             "status" => "success",
108             "message" => "Request approved"
109         ]);
110     } else {
111         echo json_encode([
112             "status" => "error",
113             "message" => "Failed to approve request"
114         ]);
115     }
116 }
```

4.1.2.3 Content Creation and Management Module:

Handles posts and comments. Posts validate title, content, and optional images, then store metadata and media. Comments support threading up to multiple levels. Editing enforces author verification and content validation; deletion uses soft-delete flags to preserve data integrity. All operations return structured objects for UI display.

```php
else if ($action === 'create_post' && $_SERVER['REQUEST_METHOD'] === 'POST') {
    $community_id = $_POST['community_id'] ?? null;
    $title = $_POST['title'] ?? null;
    $content = $_POST['content'] ?? null;

    if (empty($community_id) || empty($title) || empty($content)) {
        echo json_encode([
            "status" => "error",
            "message" => "All fields are required"
        ]);
        exit();
    }

    $image_path = null;
    if (isset($_FILES['image']) && $_FILES['image']['error'] === UPLOAD_ERR_OK) {
        $ext = pathinfo($_FILES['image']['name'], PATHINFO_EXTENSION);
        $image_path = "uploads/posts/" . uniqid() . "." . $ext;
        move_uploaded_file($_FILES['image']['tmp_name'], "../" . $image_path);
    }

    // Membership check
    $check_sql = "SELECT id FROM community_members WHERE community_id = ? AND user_id = ? AND role != 'requesting'";
    $check_stmt = mysqli_prepare($conn, $check_sql);
    mysqli_stmt_bind_param($check_stmt, "ii", $community_id, $user_id);
    mysqli_stmt_execute($check_stmt);
    $check_result = mysqli_stmt_get_result($check_stmt);

    if (mysqli_num_rows($check_result) === 0) {
        echo json_encode([
            "status" => "error",
            "message" => "You must be a member of this community to post"
        ]);
        exit();
    }


    $sql = "INSERT INTO posts (community_id, user_id, title, content, image)
            VALUES (?, ?, ?, ?, ?)";
    $stmt = mysqli_prepare($conn, $sql);
    mysqli_stmt_bind_param($stmt, "iisss", $community_id, $user_id, $title, $content, $image_path);

```

4.1.2.4 Moderation Module:

Enables reporting, review, and enforcement. Reports record content type, reporter, and reason. Moderators access queues of pending reports, review content contextually, and apply actions such as dismissal, warning, content removal, or user bans. Actions are logged with status updates.

```
interface ReportButtonProps {
  itemType: "post" | "comment" | "user" | "community";
  itemId: number;
}

const ReportButton = ({ itemType, itemId }: ReportButtonProps) => {
  const [showModal, setShowModal] = useState(false);
  const [reason, setReason] = useState("");
  const [description, setDescription] = useState("");
  const [submitting, setSubmitting] = useState(false);

  const reportReasons = [
    "Spam or misleading content",
    "Harassment or bullying",
    "Hate speech or symbols",
    "Violence or dangerous organizations",
    "Nudity or sexual activity",
    "False information",
    "Intellectual property violation",
    "Other"
  ];

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    setSubmitting(true);

    try {
      const response = await fetch(
        "http://localhost/Interconnected/backend/api/report_actions.php?action=submit_report",
        {
          method: "POST",
          credentials: "include",
          headers: { "Content-Type": "application/json" },
          body: JSON.stringify({
            reported_item_type: itemType,
            reported_item_id: itemId,
            reason: reason,
            description: description
          })
        }
      );
```

4.1.2.5 Administrative Module:

Provides platform-wide oversight. The dashboard aggregates metrics on users, communities, posts, and moderation activity. Administrators can manage users (view, suspend, ban), oversee communities (activity, moderation tools), and monitor trending content or pending reports.

```
const Admin = () => {
  const [activeTab, setActiveTab] = useState<"users" | "communities" | "posts" | "reports">("users");
  const [reports, setReports] = useState<Report[]>([]);
  const [users, setUsers] = useState<User[]>([]);
  const [communities, setCommunities] = useState<Community[]>([]);
  const [posts, setPosts] = useState<Post[]>([]);
  const [loading, setLoading] = useState(true);
  const [isAdmin, setIsAdmin] = useState(false);

  useEffect(() => {
    checkAdminStatus();
  }, []);

  useEffect(() => {
    if (isAdmin) {
      if (activeTab === "users") fetchUsers();
      else if (activeTab === "communities") fetchCommunities();
      else if (activeTab === "posts") fetchPosts();
      else if (activeTab === "reports") fetchReports();
    }
  }, [activeTab, isAdmin]);

  const checkAdminStatus = async () => {
    try {
      const response = await fetch(
        "http://localhost/Interconnected/backend/api/session-check.php",
        { method: "GET", credentials: "include" }
      );
      const data = await response.json();
      if (data.logged_in && data.user.role_u === "admin") {
        setIsAdmin(true);
      } else {
        window.location.href = "/home";
      }
    } catch (error) {
      console.error("Failed to check admin status:", error);
    } finally {
      setLoading(false);
    }
  };
```

4.2 Testing

Testing is the critical process of verifying that the implemented system meets requirements and functions correctly. Comprehensive testing at multiple levels ensures quality, reliability, and usability of the Interconnected platform.

4.2.1 Unit Testing

Unit testing examines individual components and functions in isolation to verify they behave correctly for various inputs and conditions. Each test focuses on a specific piece of functionality with known inputs and expected outcomes.

Table 4.2: User Registration Testing

S.No.	Test Case	Input	Expected Outcome	Actual Outcome	Result
1	Valid Registration	Valid username, email, strong password	User account created, confirmation message	Account created successfully, user logged in	Pass
2	Duplicate Username	Username that already exists	Error message: "Username already taken"	Appropriate error displayed	Pass
3	Duplicate Email	Email already registered	Error message: "Email already in use"	Appropriate error displayed	Pass
4	Invalid Email Format	Malformed email address	Error message: "Invalid email format"	Client and server validation caught error	Pass
5	Weak Password	Password less than 8 characters	Error message: "Password too short"	Validation prevented registration	Pass
6	Missing Required Field	Empty username field	Error message: "Username required"	Form validation prevented submission	Pass

7	SQL Injection Attempt	Username with SQL syntax	Input sanitized, account created safely or rejected	Prepared statements prevented injection	Pass
8	XSS Attempt	Username with script tags	Script tags escaped/rejected	Input sanitization prevented XSS	Pass

Table 4.3: Community Management Testing

S.No.	Test Case	Input	Expected Outcome	Actual Outcome	Result
1	Create Community	Valid name, description, category	Community created, creator becomes moderator	Community created successfully	Pass
2	Duplicate Community Name	Name already in use	Error message: "Community name taken"	Appropriate error displayed	Pass
3	Join Community	Click join on available community	User added as member, community appears in user's list	Membership created successfully	Pass
4	Leave Community	Click leave on joined community	Membership removed, community removed from list	User successfully left community	Pass
5	View Community Posts	Navigate to joined community	Posts displayed chronologically	Posts loaded and displayed correctly	Pass
6	Access Non-Member Community	Try to view private community content	Access denied message	Access properly restricted	Pass

Table 4.4: Content Posting Testing

S.No.	Test Case	Input	Expected Outcome	Actual Outcome	Result
1	Create Post	Valid title and content in joined community	Post created successfully	Post created and displayed	Pass
2	Create Post Without Membership	Post in non-member community	Error: "Must join community first"	Appropriate error displayed	Pass
3	Add Comment	Valid comment on post	Comment added and displayed	Comment successfully added	Pass
4	Reply to Comment	Reply to existing comment	Nested reply created	Reply properly threaded	Pass
5	Edit Own Post	Modify post content	Post updated successfully	Content updated correctly	Pass
6	Delete Own Post	Delete created post	Post removed from view	Post soft-deleted successfully	Pass
7	Report Content	Report inappropriate post with reason	Report submitted	Report created for moderation	Pass
8	Upload Image	Attach image to post	Image uploaded and displayed	Image properly stored and shown	Pass

Content posting and interaction testing confirmed that users can create, edit, and interact with content within their communities, with proper permissions and validation enforced.

4.2.2 System Testing

System testing evaluates the complete integrated system to ensure all components work together correctly and the system meets its functional and non-functional requirements.

Table 4.5: User Interface Testing

S.No.	Test Case	Input	Expected Outcome	Actual Outcome	Result
1	Navigation Menu	Click navigation elements	Navigate to correct pages	Navigation works correctly	Pass
2	Single-Community View	Switch between communities	Only one community displayed at a time	Proper single-community display	Pass
3	Responsive Design Desktop	View on desktop (1920x1080)	Layout adapts properly	Interface displays correctly	Pass
4	Community Switching	Navigate between joined communities	Smooth transition, content loads	Community switching works smoothly	Pass
5	Error Messages	Trigger validation errors	Clear, helpful error messages	Errors displayed appropriately	Pass
6	Loading States	Navigate to data-heavy pages	Loading indicators shown	Loading states display properly	Pass

Table 4.6: Admin Interface Testing

S.No.	Test Case	Input	Expected Outcome	Actual Outcome	Result
1	Access Admin Dashboard	Login as admin	Admin dashboard accessible	Dashboard loads with statistics	Pass
2	View Platform Statistics	Access dashboard	User, community, post counts displayed	Statistics shown correctly	Pass
3	Manage User Account	Suspend user account	User cannot login	Account suspension works	Pass
4	Review Reported Content	Access moderation queue	Reported items displayed with context	Reports shown properly	Pass
5	Remove Inappropriate Content	Delete reported post	Post removed from platform	Content removal successful	Pass
6	View Community Details	Access any community	Full community information visible	Admin can view all communities	Pass
7	Access Non-Admin as Regular User	Regular user tries admin URL	Access denied	Proper authorization enforced	Pass

CHAPTER 5
CONCLUSION AND FUTURE RECOMMENDATIONS

5.1 Lessons Learnt/Outcome

The development of the Interconnected platform provided valuable learning experiences and insights across multiple aspects of web application development, user experience design, and project management.

Technical Skills Development:

Working with React.js provided deep understanding of component-based architecture and state management. Building reusable components and managing application state through hooks significantly improved frontend development skills. The experience highlighted the importance of properly structuring components and managing data flow in complex applications.

Backend development with PHP reinforced the importance of secure coding practices, particularly in handling user authentication, input validation, and database operations. Implementing prepared statements for all database queries and properly hashing passwords became second nature, emphasizing that security must be built in from the beginning rather than added later.

Database design and optimization proved more complex than initially anticipated. Creating an efficient schema required careful consideration of relationships, normalization, and query patterns. Adding appropriate indexes significantly improved performance, demonstrating the importance of thinking about scalability from the start.

User Experience Insights:

The single-community viewing approach, while central to the platform's concept, required careful implementation to feel natural rather than restrictive. User testing revealed that clear visual indicators of which community is currently active and easy switching between communities were essential for user satisfaction.

Simplicity in design proved more challenging to achieve than complexity. Every feature addition had to be carefully evaluated against the platform's goal of reducing distraction

and promoting focus. The iterative process of removing unnecessary elements taught the value of restraint in interface design.

Project Management Lessons:

Following the Waterfall methodology provided clear structure and milestones but also highlighted the importance of flexibility within structured approaches. While the linear progression from requirements through design to implementation worked well for this project, some iterative refinement was necessary based on discoveries during implementation.

Time estimation for development tasks improved throughout the project. Initial estimates often proved optimistic, particularly for features involving complex database queries or intricate user interactions. Building buffer time into schedules became an important lesson for future projects.

Development Process Insights:

Testing early and often proved invaluable. Writing test cases alongside development rather than waiting until the end helped identify issues when they were easier to fix. Unit testing individual components before integration saved significant debugging time later.

Documentation during development rather than after completion made the final report much easier to write and ensured important decisions and rationale were captured. Maintaining clear code comments and documenting API endpoints throughout development proved essential.

Version control became indispensable for managing changes and experimenting with new features without risking working code. Learning to write meaningful commit messages and structure commits logically improved code management significantly.

5.2 Conclusion

The Interconnected platform successfully demonstrates an alternative approach to social networking that prioritizes meaningful engagement over algorithm-driven content. It provides a community-first environment allowing users to focus on one community at a time, reducing information overload and promoting deeper interaction.

All core functional requirements are met: user registration and profile management, community discovery and participation, content creation and interaction, moderation tools

for community managers, and administrative oversight. The React.js frontend delivers a responsive interface, while the PHP backend ensures secure authentication, authorization, and data handling, supported by a structured MySQL database. Security best practices—including password hashing, input validation, prepared statements, and session management—protect user data.

Testing confirmed reliable performance, correct functionality, and acceptable response times. The project validates that socially responsible, community-focused platforms are technically feasible and conceptually valuable, laying a foundation for further enhancements while maintaining user wellbeing.

5.3 Future Recommendations

Enhancements can expand functionality while supporting the platform's focus on intentional engagement:

High Priority:

- Real-time notifications via WebSockets or server-sent events.
- Enhanced search with full-text search, filters, and relevance ranking.
- Richer user profiles with activity history, badges, and customizable themes.
- Content-rich editor with formatting, lists, links, code blocks, and embedded content.
- Native mobile applications for improved accessibility and offline use.

Medium Priority:

- Private and group messaging with notifications.
- Advanced bookmarking with collections, tags, and sharing.
- Community analytics for moderators (engagement metrics, active users).
- User blocking/muting and harassment reporting.
- Post scheduling and improved community discovery features.
- Multimedia support including video, audio, and documents.
- User verification for trust and credibility.

Lower Priority:

- Community events, polls, and wiki-style knowledge bases.
- Internationalization and accessibility improvements.

- Technical Improvements:
- Performance optimization (caching, indexes, CDNs, lazy loading).
- API development for third-party integration.
- Analytics and monitoring dashboards.
- Backup, recovery, and scalability enhancements.
- Community & Moderation Features:
- Advanced moderation tools (spam detection, reputation systems).
- Community customization (themes, post types, rules).

Prioritizing features that reinforce focused, meaningful engagement ensures the platform grows without compromising its core philosophy.

REFERENCES

[1] J. Chan, F. Choi, K. Saha, and E. Chandrasekharan, “Examining Algorithmic Curation on Social Media: An Empirical Audit of Reddit’s r/popular Feed,” arXiv preprint, arXiv:2502.20491, 2025.

[2] O. El Malki, M. Aubin Le Quéré, A. Monroy-Hernández, and M. H. Ribeiro, “BONSAI: Intentional and Personalized Social Media Feeds,” Proc. ACM SIGCHI, 2025.

[3] A. M. Roffarello and L. De Russis, “Achieving Digital Wellbeing Through Digital Self-control Tools: A Systematic Review and Meta-analysis,” ACM Trans. Comput.-Hum. Interact., vol. 30, no. 4, 2023.

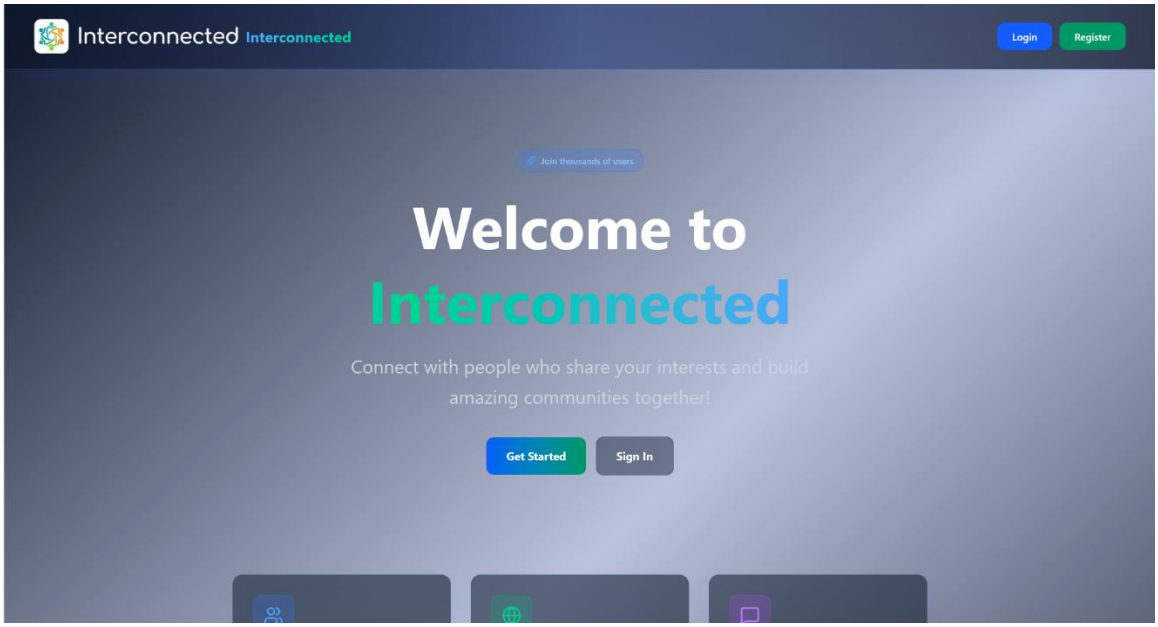
[4] K. Davis et al., “Supporting Teens’ Intentional Social Media Use Through Interaction Design,” Proc. Interaction Design and Children, 2023.

[5] S. Jhaver, S. Frey, and A. X. Zhang, “A Design Space of Multi-Level Governance in Online Social Platforms,” Proc. ACM Hum.-Comput. Interact. (CSCW), 2023.

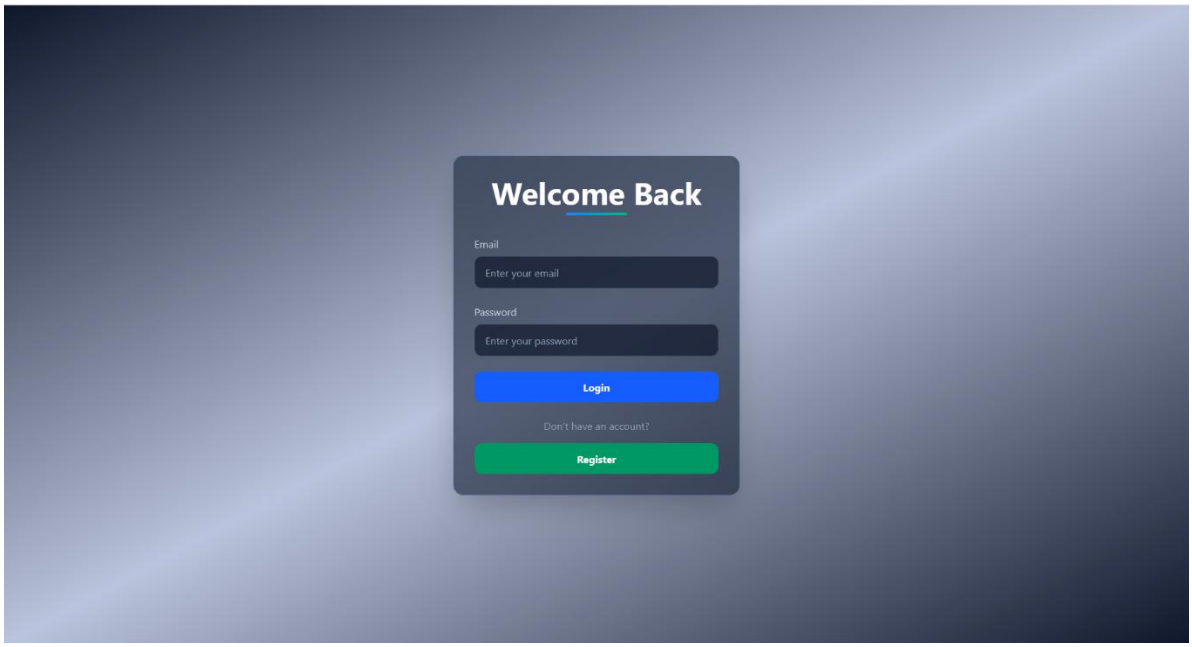
[6] J. Clear, Atomic Habits: Tiny Changes, Remarkable Results. New York, NY, USA: Avery, 2018.

APPENDICES

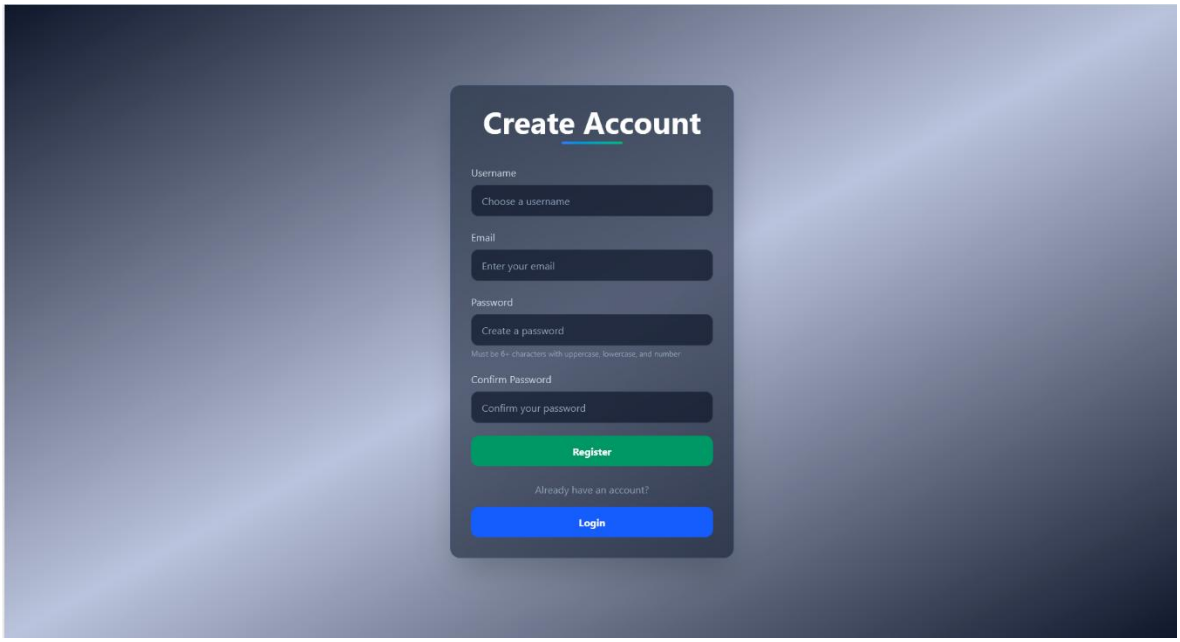
1. Landing Page



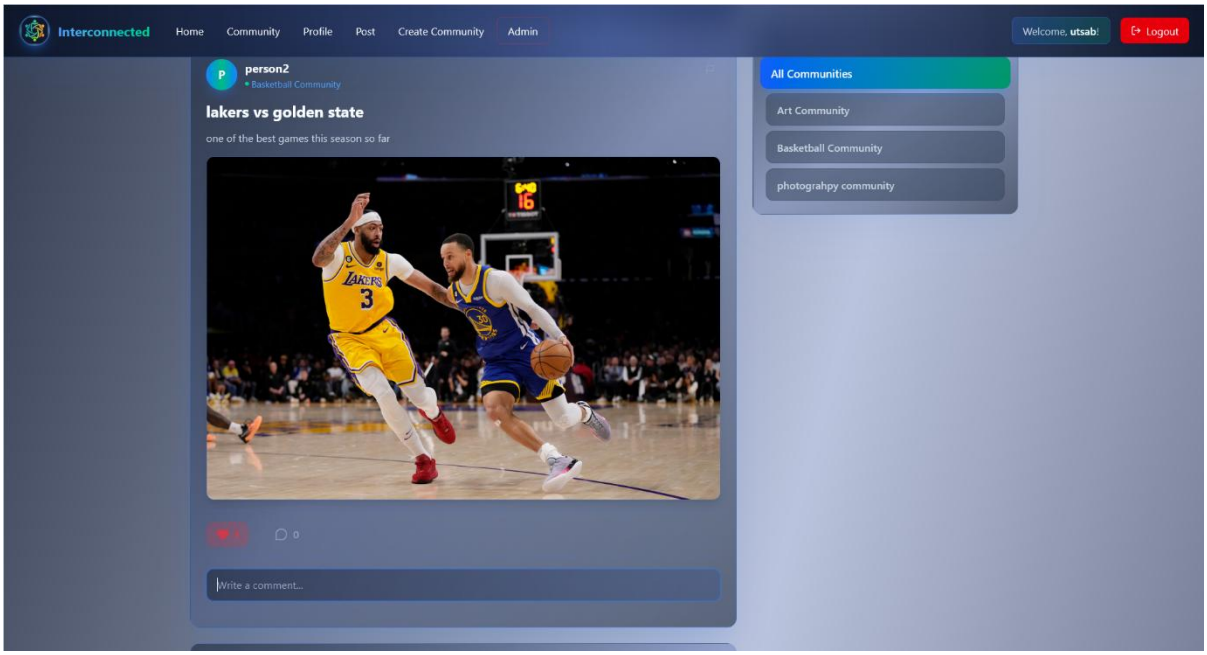
2. Login Page

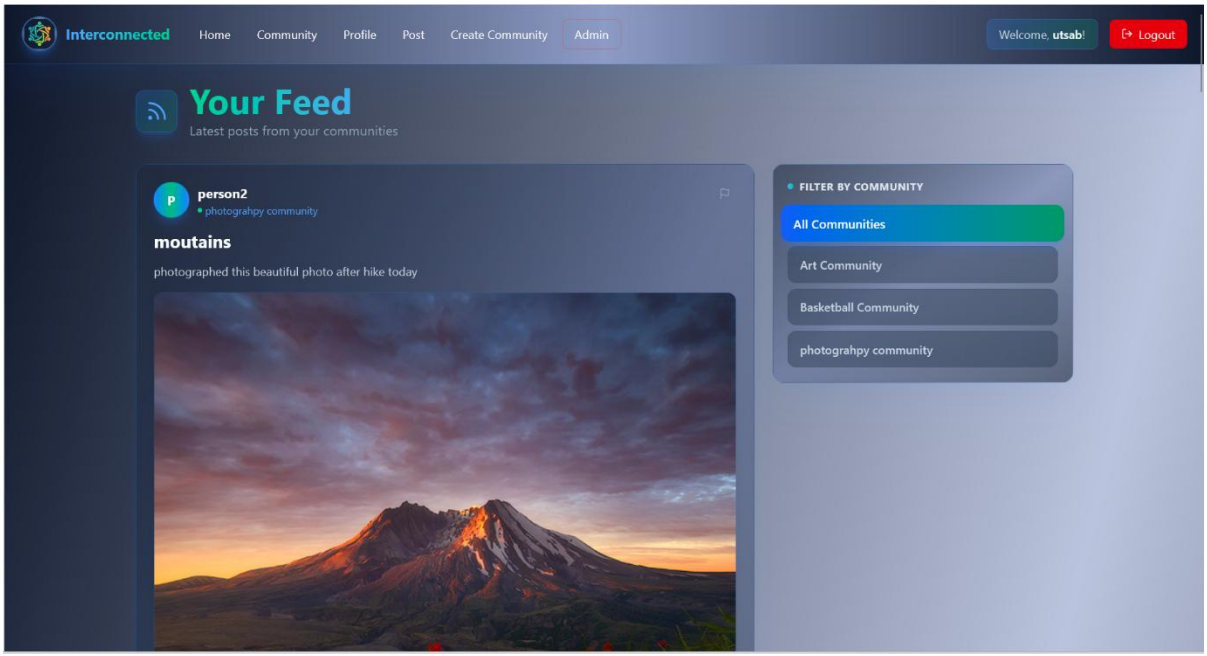


3. Registration Page

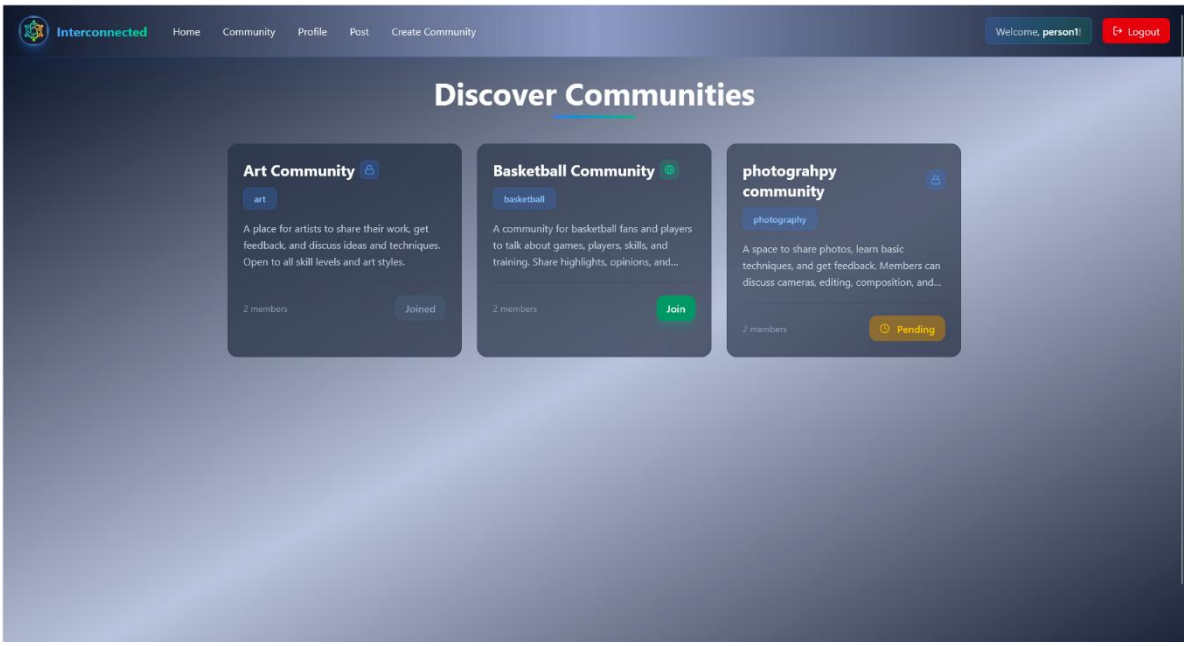


4. Home/Feed Page

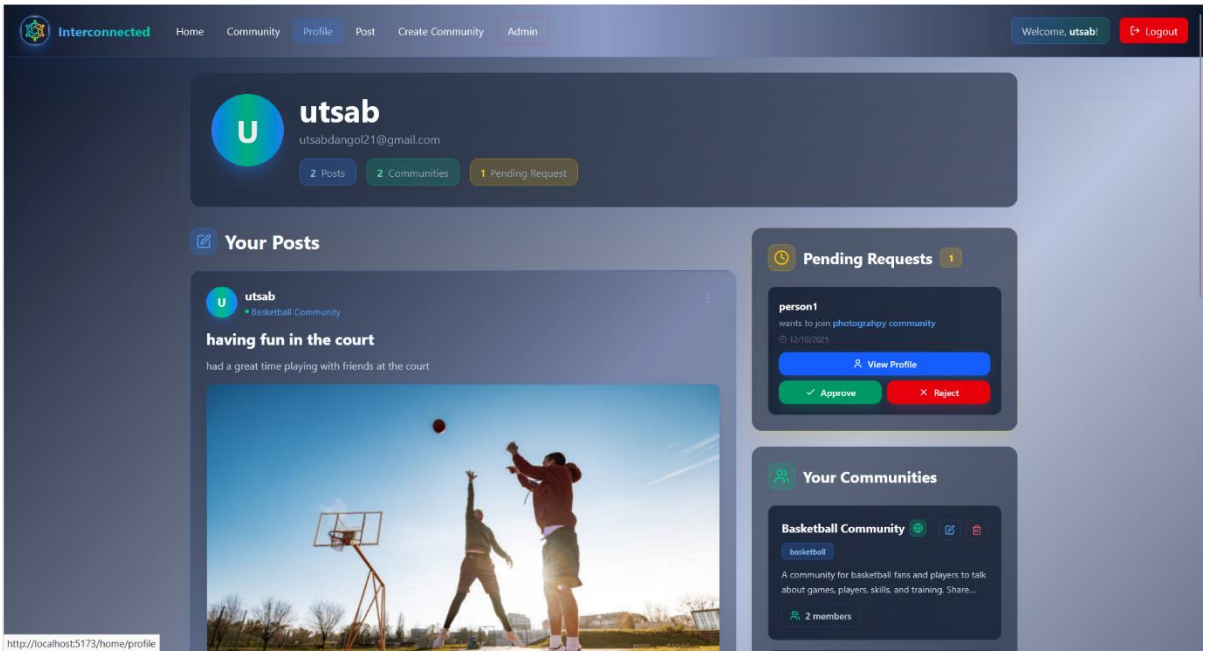




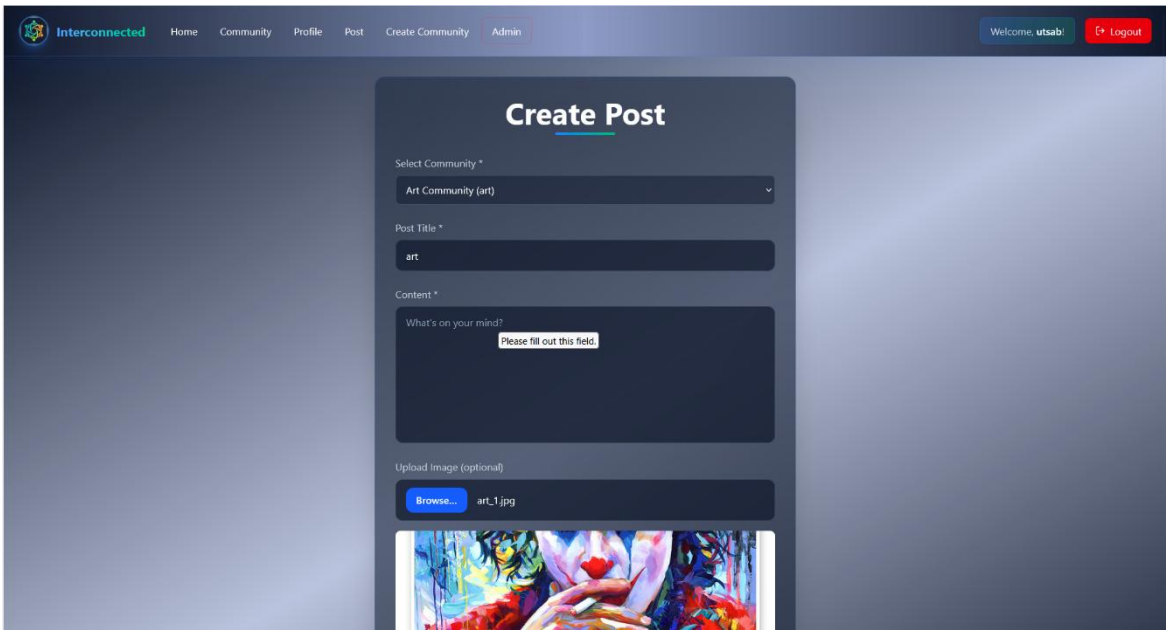
5. Community Discovery Page



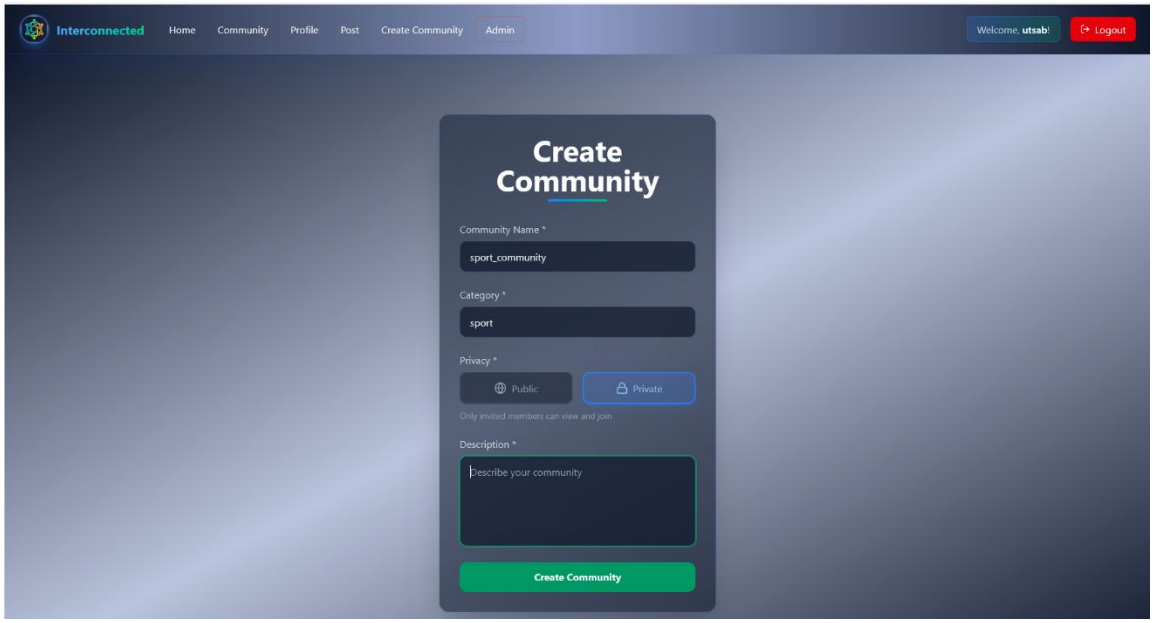
6. Profile Page



7. Post Creation Page



8. Community Creation Page



9. Admin Panel

